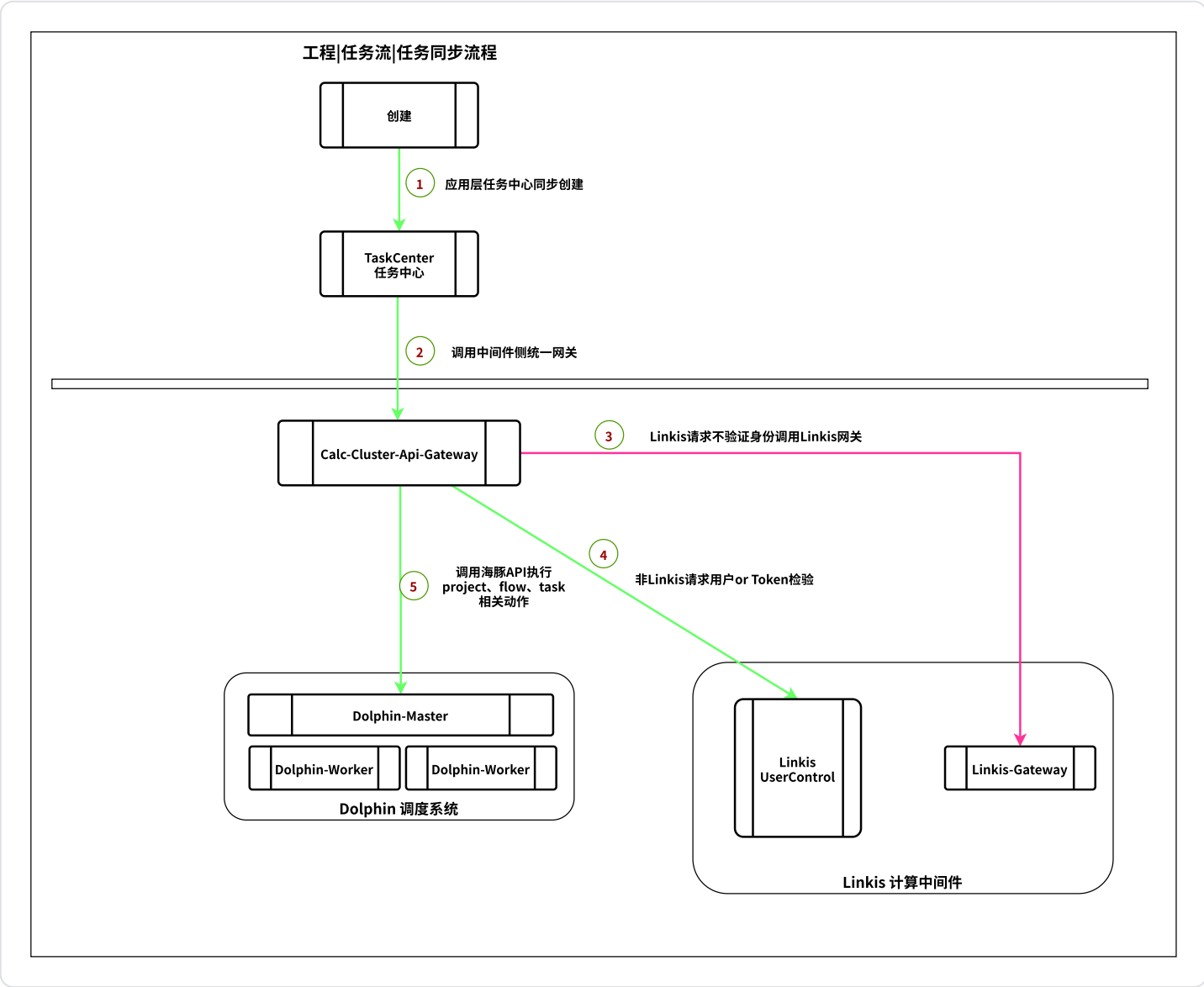


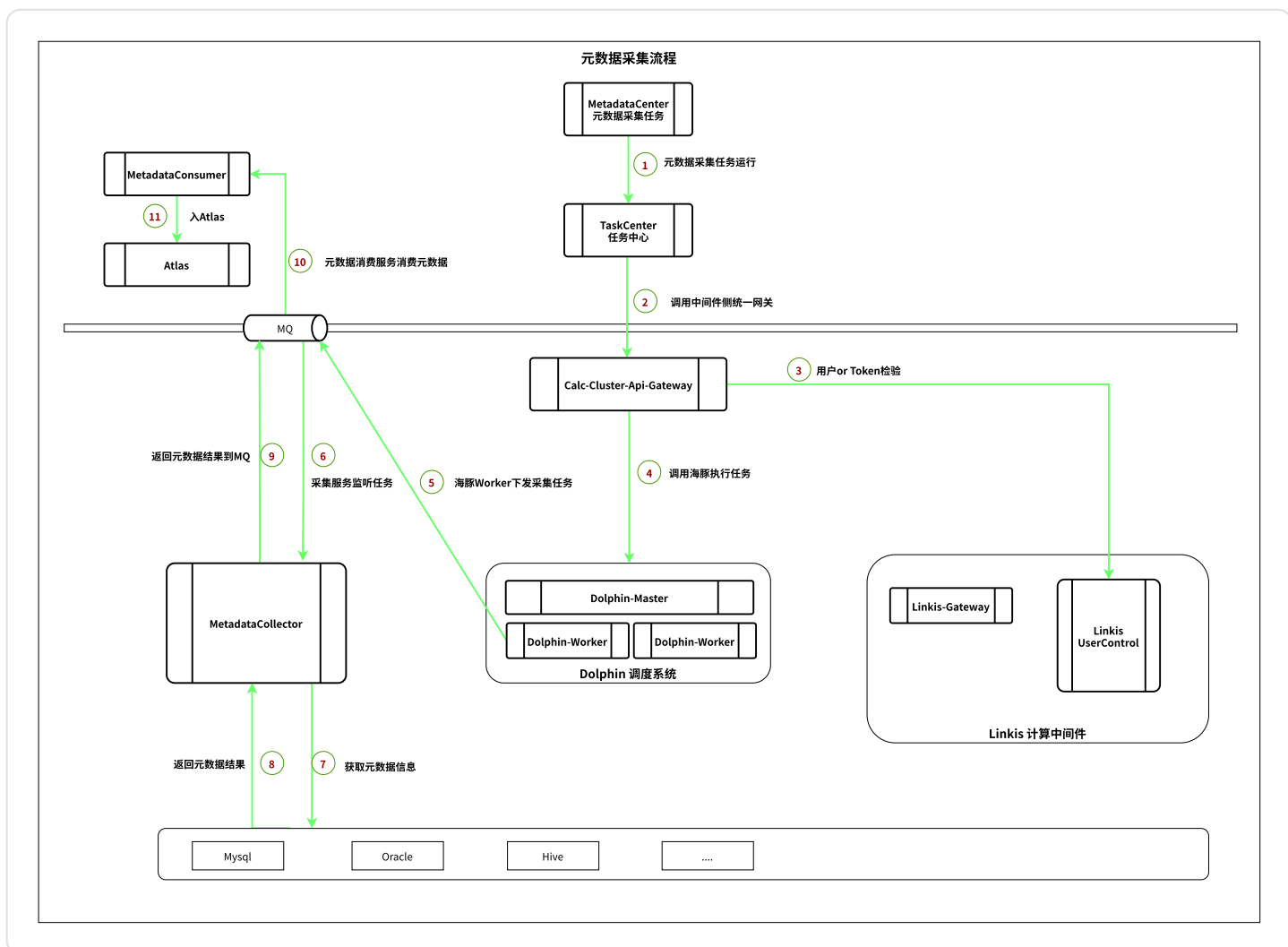
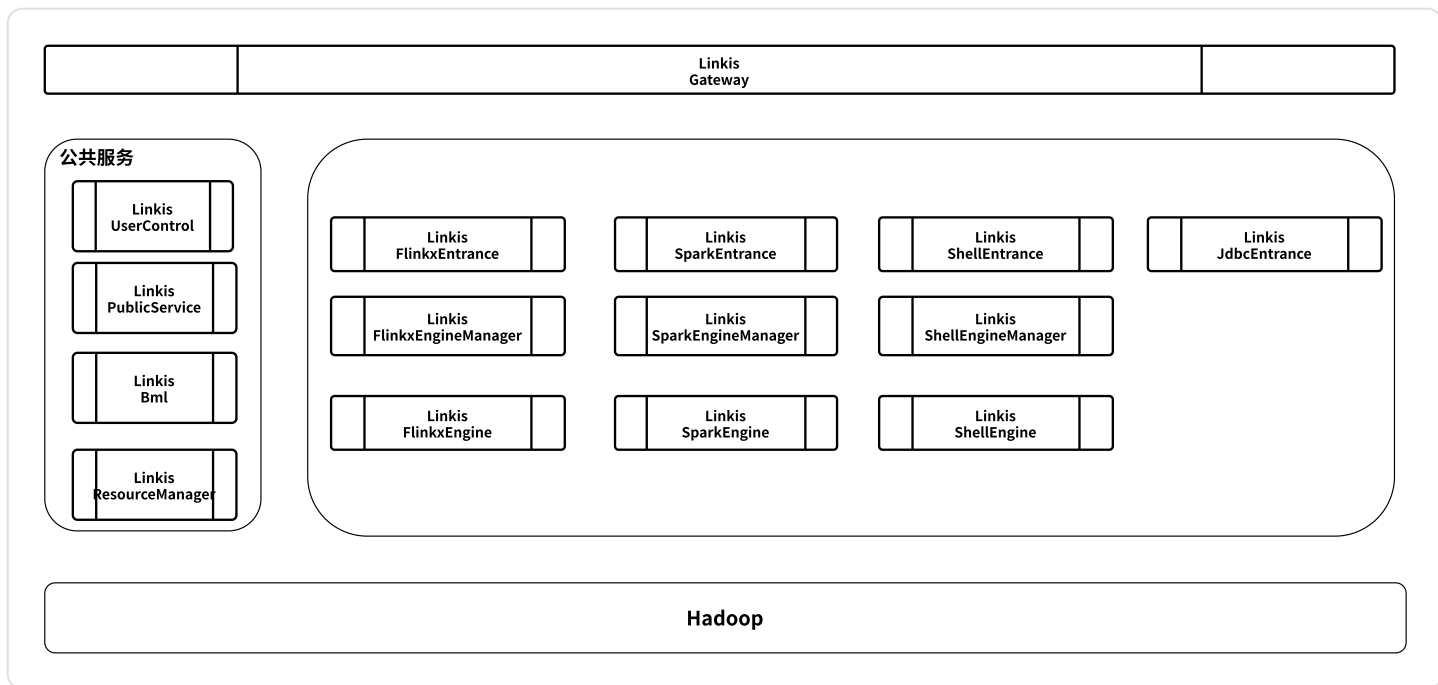
统一中间件专项

一、中间件现状分析

1.1 中间件现状



质量集成开发任务调用流程



1.2 目前中间件存在的问题

代码的组织结构

- jar包冲突严重
- module作为共享包的设计，不易于调试和构建镜像

原有的设计在K8S环境下显得过于复杂度，资源占用过高

组件/机制	原有设计方式和作用	在K8S环境下原有设计的问题	K8S下的解决方案
EngineManager	管理一个计算节点的资源。在计算节点上启动和停止引擎	资源管理功能失效	
FlinkxEngine	生成JobGraph，提交到Yarn上。FlnkxEngine本身不运行任务	1、升级flink版本困难 2、对于flink jar on yarn时，通过命令行启动，实际上在flinkxEngine内部又启动了一个Flink客户端进程。导致FlinkxEngine的内存必须设置在2G	利用容器隔离的特点。 1、OnYARN用命令行的方式提交任务。 2、OnK8S拼装成yaml提交至FlinkOperator。或用flink原生支持k8s特性，提交任务至k8s。
ShellEngine	引擎复用。收集shell执行的日志发送至entrance。	ShellEngine为一个SpringBoot程序。 1、启动过慢。 2、资源额外占用：一个ShellEngine至少占用500MB内存。而Shell本身是一个简单的任务	1、不需要启动额外的jvm进程 2、日志收集可通过轻量化的sidecar实现
共识机制	引擎状态变更，通过rpc广播方式。	实现很复杂。且出了问题不好排查。	通过zookeeper、etcd或使用k8s的operator扩展实现。
服务发现	eureka	难以利用k8s的现有机制，实现平滑发版	用service实现
RPC通信	基于feign封装的rpc框架	1、与eureka强绑定 2、rpc的url无法明确表达调用的是什么方法，不易做链路跟踪 3、体系封闭，不易扩展非该rpc框架外的服务	结合service，用restful实现

状态存储在服务进程内部，无法平滑发版

- Entrance
 - 作业的队列。服务重启后会导致未调度的作业不再被调度；运行的作业状态查不到；正在运行的作业无法被停止（常驻的shell任务。on yarn通过publicservice中的状态轮训器实现）
- EngineManager
 - EngineManager正常关闭时，会把所有的engine shutdown。
 - Engine的列表。用于实现Engine的复用
- Engine
 - Engine中持有Entrance的地址。向Entrance发送任务的日志。
- ResourceManager
 - EngineManager的列表。ResourceManager重启后，会丢失EngineManager的状态。此时EngineManager向ResourceManager申请资源时，会提示该EngineManager没有在ResourceManager中注册，导致启动引擎失败。

4. 没有为常驻服务设计

线上遇到的问题

enginemanager停止时会kill管理的所有engine

造价成本和数字服务同时使用数据中台，数字服务把yarn的资源占完了，导致成本任务运行失败

出现异常后，异常信息不明确（很多异常都是提示资源不足，排查实际异常耗费时间长）

Pod由于OOM被k8s kill了，没有体现在日志中

1.3 linkis依赖的存储

Redis mysql ElasticSearch 对象存储等

1. Redis

- a. 用户中心，账户、密码等

2. Mysql (2个库)

- a. Linkis 元数据持久
- b. 异常关键字持久

3. ElasticSearch

- a. 启动日志，Entrance 直接writer
- b. 运行日志，filebeat 或 Log4j适配器直接写入（Kafka + Logstash）

4. 对象存储

- a. 物料库
- b. 发布附件文件

1.4 linkis对外提供的接口

概况：

从各个服务看：

规范建模、数据集成、数据服务等：	获取元数据、操作BML
海豚worker：	提交任务、kill任务
海豚任务助手：	获取任务状态信息 status
数据质量：	拉取质量任务结果
迁移助手？	

具体接口情况：

元数据接口wiki：<https://km.glodon.com/pages/viewpage.action?pagelId=180052380>

A. 测试连接、规范建模、数据集成、数据服务（元数据获取相关）：

测试连接、数据库实例的元数据信息（库，表，字段，索引，主键）、执行ddl

- 测试连接：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/connection
- 获取数据库列表：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/databases
- 获取数据库表列表：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/tables
- 获取数据库表列表-全：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/tableList
- 获取数据表信息：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/table/info
- 获取数据库表字段列表：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/columns
- 获取数据库表主键列表：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/primaries
- 获取数据库表字段信息：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/column/info
- 获取数据索引列表：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/indices
- 获取数据表索引keys：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/keys
- 获取数据表分区列表：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/partitions

- 查询starRocks执行状态：
http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/starRocks/execute/state
- 获取数据库表信息-hive：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/table/props
- 获取索引信息-mongo：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/mongo/index
- 获取最后一条数据-mongo：
http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/mongo/last/document
- 随机获取指定条数据：
http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/mongo/multiple/column
- 执行sql：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/execute/sql
- 获取示例文档：http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/document

B. 数据开发、数据集成任务、数据查询、数据编目预览、数据质量、个人空间

- spark sql 任务
- jdbc 任务
- python 任务
- shell 任务
- Flinkx 任务
- Flink stream sql 任务
- presto
- spark scala
 - 登陆：
 - url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/user/login
 - 请求方式: POST
 - 参数

 {"userName": "lius-f", "password": "1234566", "nonce": "c393d8a3ba0a42ae9d4c55c33d9a125d", "timestamp": "1609747531052", "sign": "kybbDxjCpQANTV6FNBzyQtQTYbhg7+uYH0+pHqs4J64=", "appKey": "gFdjlmaaj"}

- 返回

```
 {  
  "method": null,  
  "status": 0,  
  "message": "Login successful(登录成功)",  
  "data": {  
    "token": "bVB4Qal--A2vSAI5gNdYebTIpl0Ib2i1mkPqJ4yGh6CYnyl0_Cwl-fhc6A_pO1LI"  
  }  
}
```

- 用户注册接口：

- url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/user/register
- 请求方式：POST
- 参数：

```
 {"appKey":"gFdjlmaj","nonce":"2222","password":"123456","sign":"b4gnlr4B63pgGPX8Zzd0O6Zli2vOwnJppoqpCqMF8xU=","timestamp":"1661339900364","userName":"chengwq"}
```

- 返回

```
 {  
  "method": "register",  
  "status": 0,  
  "message": "register success",  
  "data": null  
}
```

- 以上任务统一入口接口：

- http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/entrance/execute/
- 请求方式：POST

- 参数:

```
 {  
    "user": "",  
    "executeCode": "",  
    "formatCode": false,  
    "creator": "",  
    "engineType": "",  
    "runType": "",  
    "engineTypeStr": "",  
    "runTypeStr": "",  
    "scriptPath": "",  
    "params": "",  
    "source": "",  
    "execID": "",  
    "taskID": "",  
    "dolphinProjectId": "",  
    "dolphinProcessId": "",  
    "dolphinProcessInstanceId": "",  
    "dolphinTaskInstanceId": "",  
    "dolphinRetryTaskInstanceId": "",  
    "dolphinTaskName": "",  
    "dolphinTaskStatus": "1"  
}
```

- 返回

```
 {  
    "user": "",  
    "executeld": "",  
    "taskId": "",  
}
```

```
"jobId": "",
"applicationId": "",
"state": "",
"checkPointPath": "",
"isSucceed": ""
}
```

○ kill接口

- url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/flinkxcontrol/kill
- 请求方式: POST
- 参数:

```
 {
  "user": "",
  "execID": "",
  "taskID": ""
}
```

○ 状态接口:

- url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/entrance/{id}/status
- 请求方式: GET
- 参数

```
 user 用户
execID 执行id
taskID 任务id
```

○ 质量拉取任务结果 - 根据taskId获取任务信息

- Url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/jobhistory/{id}/get
- 请求方式: GET
- 参数



user 用户

execID 执行id

taskID 任务id

- 响应



JobInfoResult jobInfoResult? ? ? 待日志打出log

- 质量拉取任务结果 - 根据任务信息获取结果列表(路径)
 - Url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/filesystem/getDirFileTrees
 - 请求方式: GET
 - 参数:



JobInfoResult jobInfoResult? ? ? 待日志打出log

- 质量拉取任务结果 - 根据路径获取结果
 - Url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/filesystem/openFile
 - 请求方式: GET
 - 参数:



path 路径

user 用户

page 页号

pageSize 每页大小

C. 资源管理、迁移助手 (bml 封装obs) 、

- 上传资源:
 - url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/bml/upload
 - 请求方式: POST
 - 参数:



system String

来自哪个子系统的资源，可以统一为gdmp，或者 按照gdmp模块来划分，

eg: datasource, batchdev

resourceHeader String

物料库rootdir+"/"+sechme+ "/" + user + "/" + dateStr + "/" + resourceHeader
+ "/" + fileName

isExpire String

是否过期，0表示不过期，1表示过期

expireType String

不过期的情况下设置为 null

expireTime String

不过期的情况下设置为 null

maxVersion int

默认为10，指保留最新的10个版本, 如果想保留跟多的版本，可以设置较大数值

- 更新资源文件

- url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/bml/updateVersion
- 请求方式: POST
- 参数



resourceId String

资源id

formDataMultiPart FormDataMultiPart (key: file)

更新的文件

- 下载资源文件

- url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/bml/download
- 请求方式: get
- 参数



resourceId String

资源id

version String

资源版本id

- 删除资源

- url: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/bml/deleteResource
- 请求方式: POST
- 参数



resourceId String

资源id

D. 数据资产

数仓类数据统计, hive元数据统计接口

- 获取hive统计信息: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/statistics

E. 数据安全任务 (linkis 元数据接口)

- 获取数据库表列表-全: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/tableList
- 获取数据库表字段列表: http://linkis-gdmp-dev.glodon.com/api/rest_j/v1/dsm/meta/columns

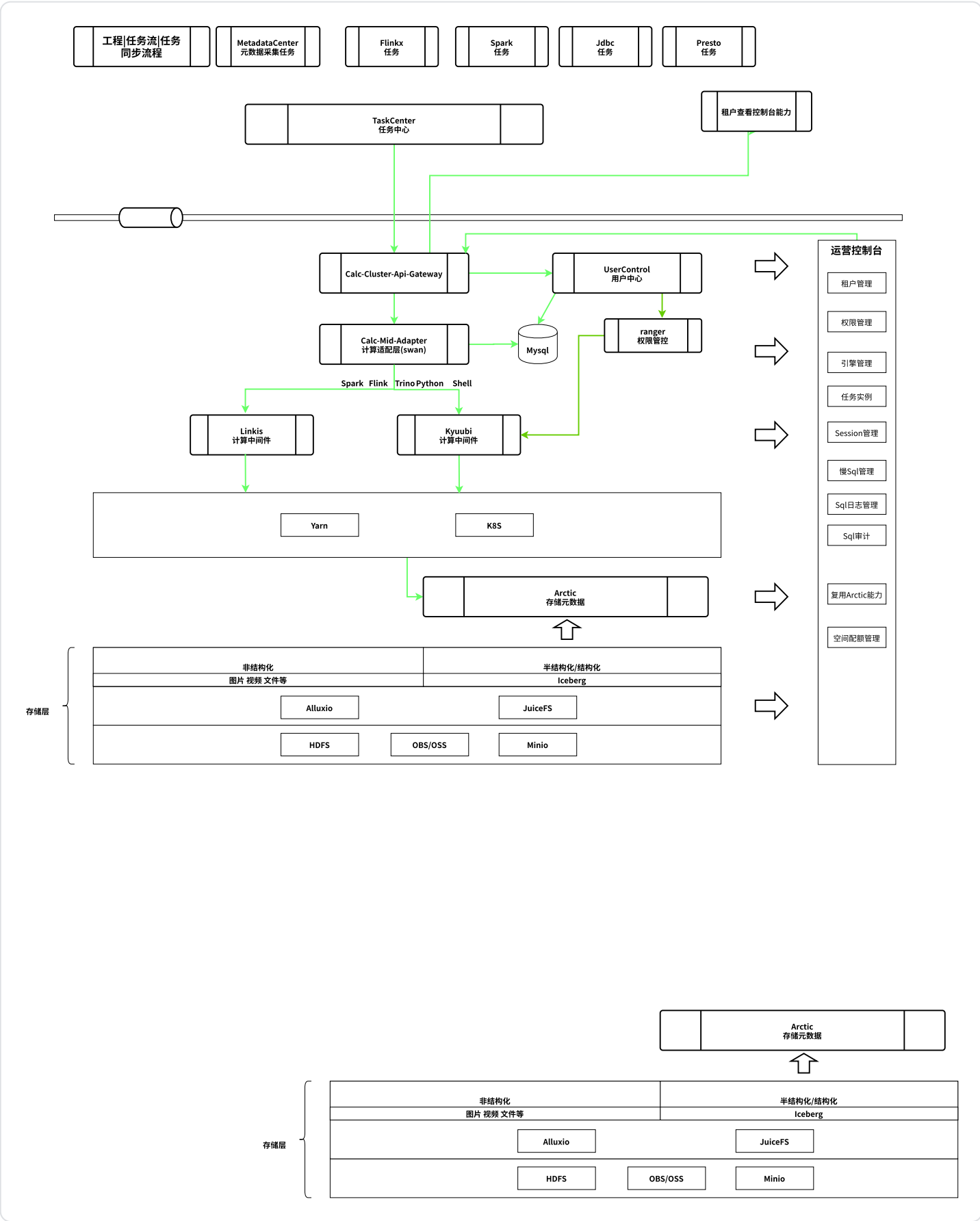
二、调研现阶段开源中间件

Kyuubi (其架构、优势/缺点、支持spark/flink/trino、支持多租户、支持on yarn/k8s等特点)

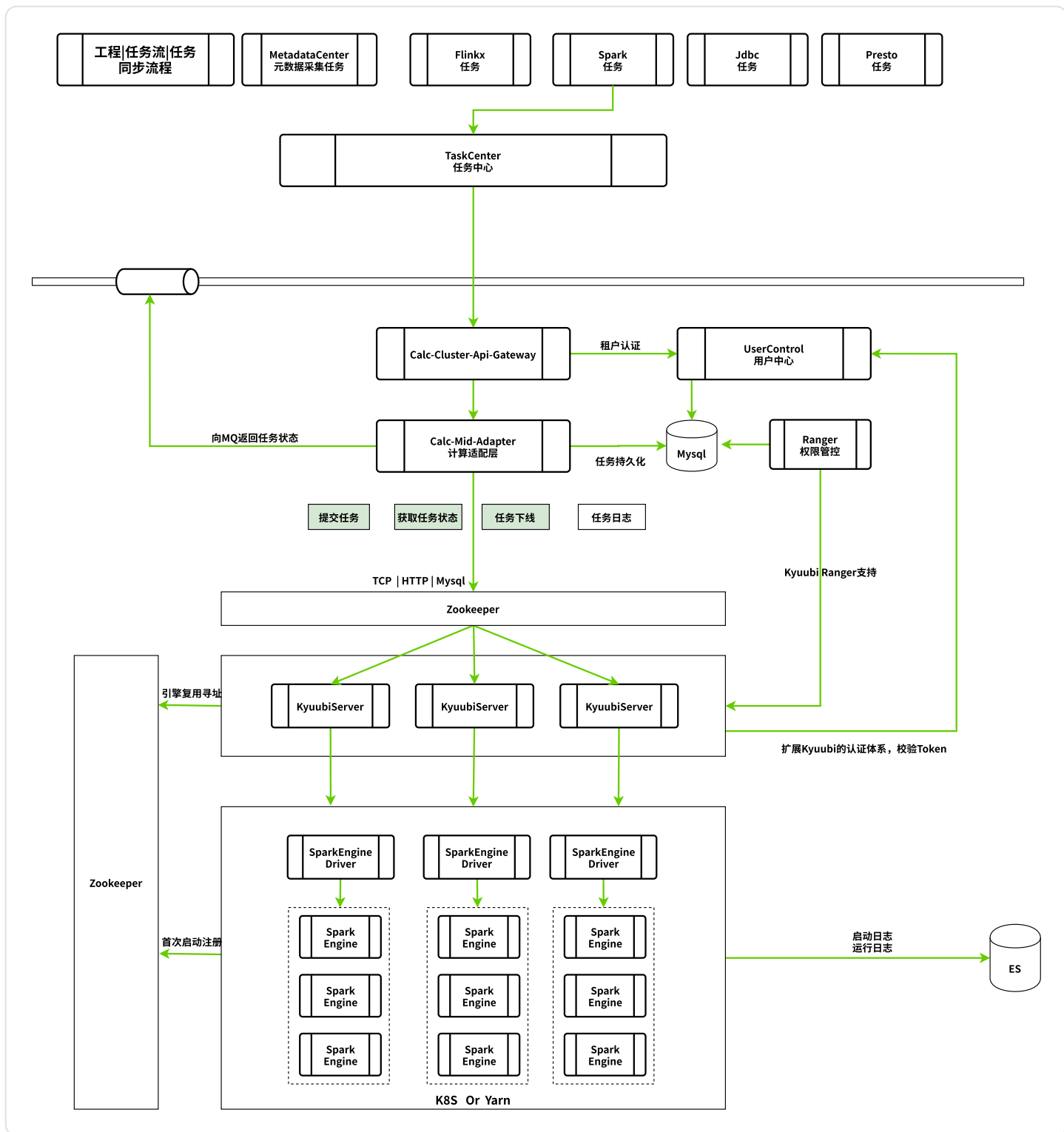
三、升级、部署(应用层无感知)

应用层怎样与kyuubi无缝对接

替换原linkis相关能力, 逐步弱化linkis能力

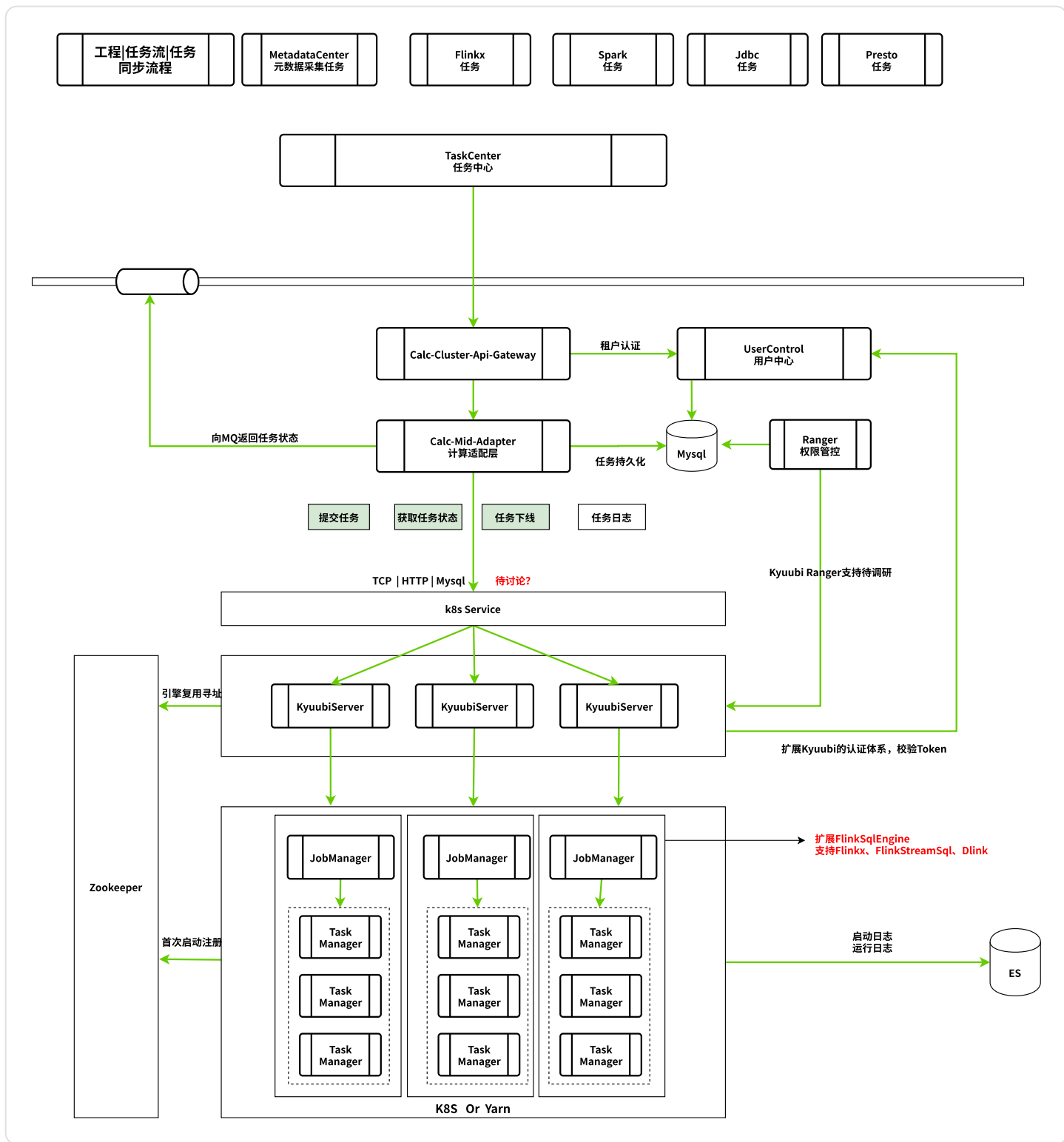


Kyuubi 替换Spark 任务



Flinkx FlinkStreamSql任务

方案1：让Kyuubi支持flinkx和flinkStreamSql任务

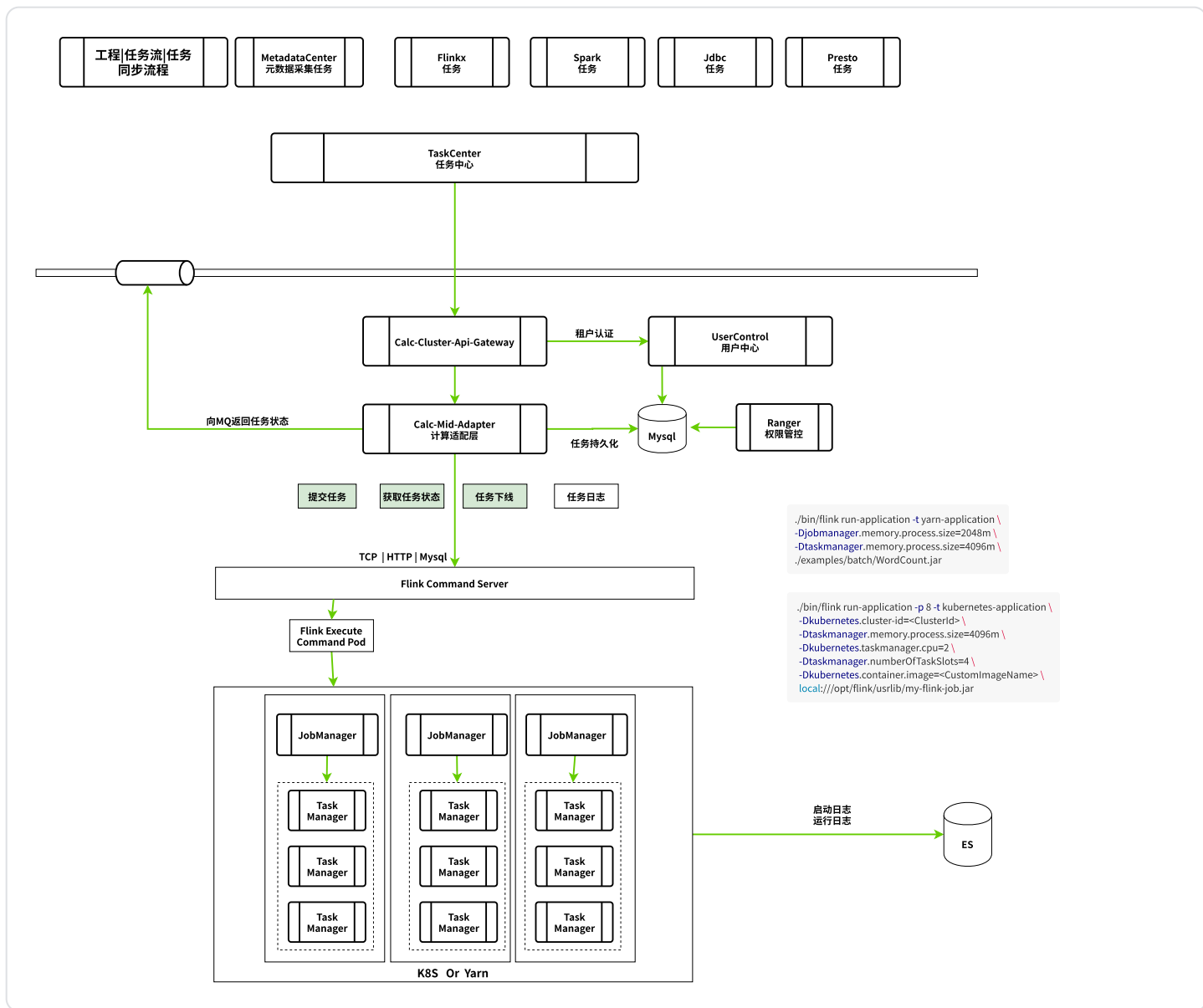


方案2：弹pod，该pod的启动命令就是 flink原生命令行，启动任务

为什么不能在FlinkCommandServer中执行flink命令行？

flink命令行会起一个java进程，消耗FlinkCommandServer的资源。不可控。

FlinkExecuteCommandPod只是执行一个提交命令，启动应该很快，不需要作为引擎复用。

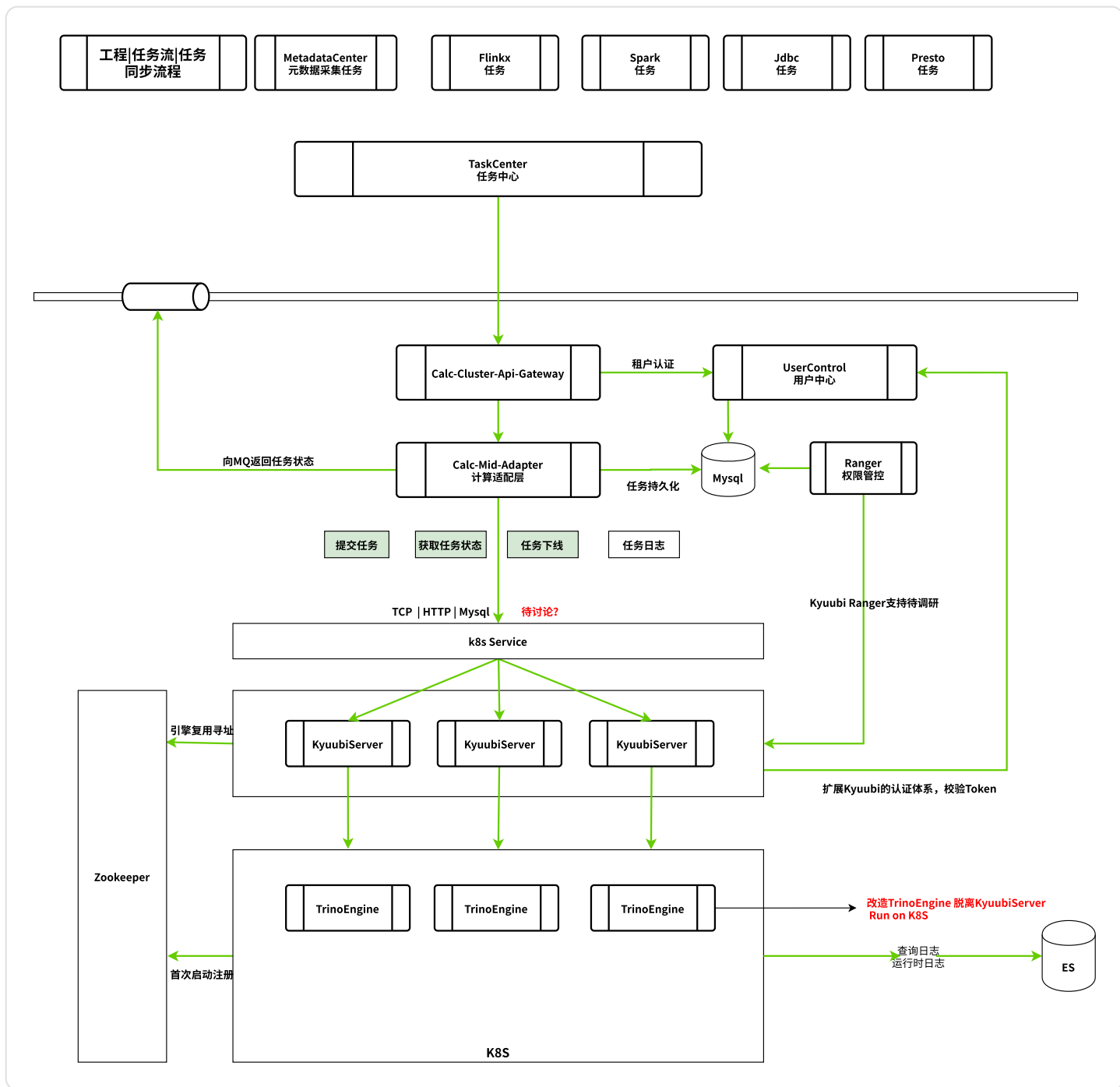


Shell类任务

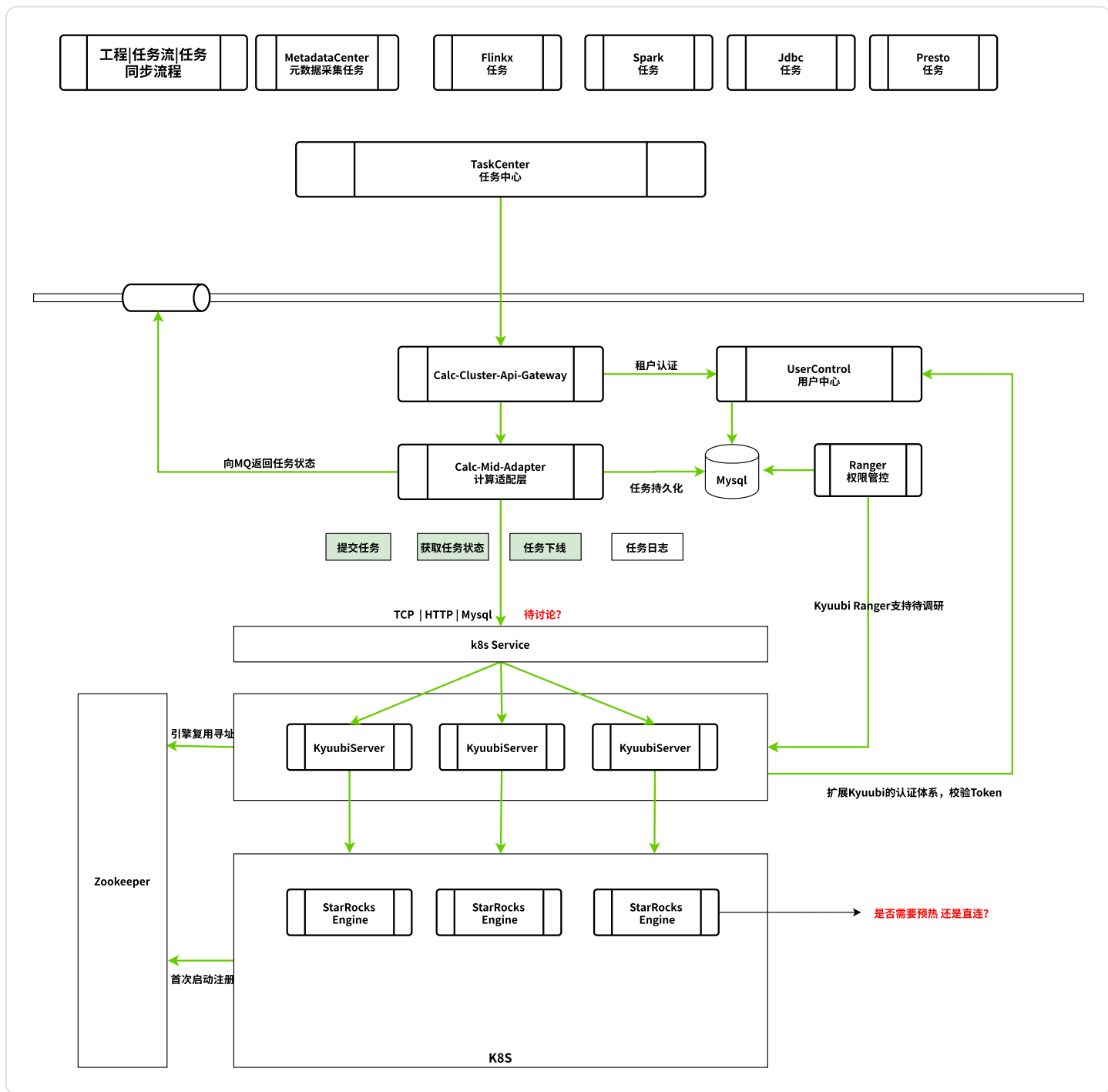
弹pod，该pod的启动命令就是shell的脚本

Kyuubi 替换Trino任务

原生Trino不支持动态catalog。需测试KyuubiServer与openlookeng的兼容性。



Kyuubi 替换 StarRocks任务



四、待议内容

1.linkis的问题，链路长、状态等，换成kyuubi后能解决哪些问题，实现成本有多高？

2.应用层与kyuubi客户端的连接是短连接异步，怎样获取已提交任务的“状态”、进度等（kyuubi client和server是长连接，但是应用层与其client连接是短连接）

适配层维护应用层的任务实例，状态，进度，日志，应用层无需感知后端kyuubiServer的存在，应用层直接和适配层交互，不需要和后端中间件交互。自然就不存在状态的问题。

3. dlink怎么融合进新的体系；适配层加一个路由到dlink分支，与海豚、kyuubi同级；

a. kyuubi扩flinkx streamsql dlink

倾向于扩展kyuubi的flink引擎，兼容flinkx 和flinkStreamSql及Dlink的任务，尽管实现成本会高些，但是保留完成Kyuubi 统一sql 网关的设计原则，同时租户，权限管控可以很容易实现；

b. dlink的元数据体系考虑合并适配层，用于权限管控

4. dlink和现有元数据体系结合

dlink的技术元数据和适配层融合，和租户体系结合起来

5. 如果接入kyuubi，方案尽可能完整 任务资源管控rm和uc关系 物料库怎么去管理，跟用户关系 如果平级扩dlink、shell、python等扩其他场景组件 权限管控咋弄（kyuubi还是适配器权限管控，替换原linkis的管控能力） 具体咋扩，kyuubi上扩还是平级扩，应用层什么场景用什么组件(其机制) 轻量化能轻到什么程度 调用链路过长 数据查询（加密、脱敏、安全方面）

6.

场景越来越多，无限制的扩组件能力，范围、管控(资源、权限控制等)问题

1. 采用技术收敛策略，尽量引导用户基于sql的方式实现。

2. 技术能力往sql方向靠齐

7. 节奏，时间节点（先定方向） 大方向 - 没问题 细节方向 - 比如状态

8. 落地价值、投入的资源（后期维护资源、扩新组件场景代价、任务运行稳定情况）

9. 应用层脱敏、加密（ranger能力）

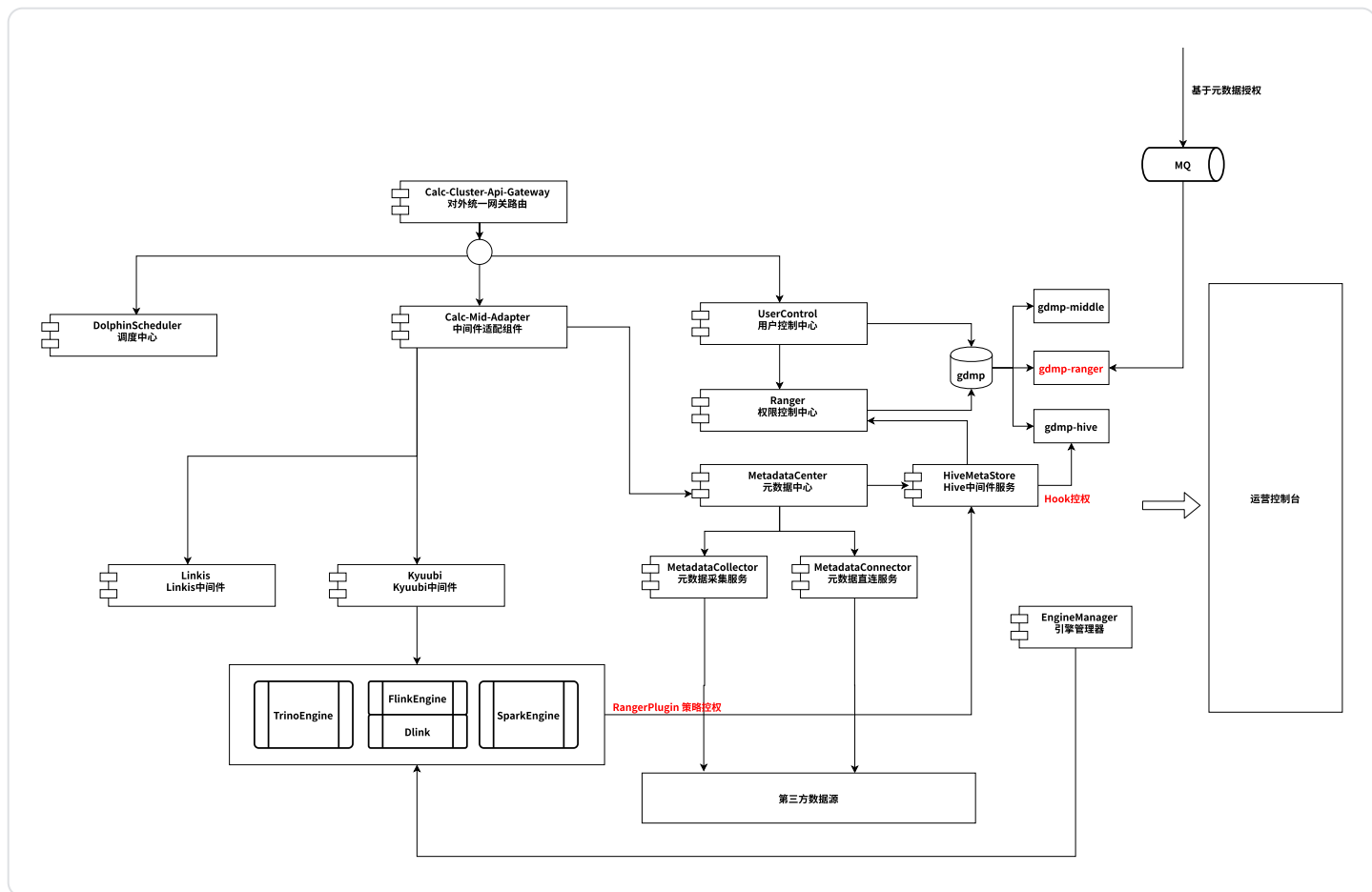
借助于适配层的租户体系设计，利用kyuubi的 h2 sql网关能力和ranger结合起来，能够轻松实现加密、脱敏、库表列权限等

5.1、方案评审后问题记录

1. 适配层中哪些地方会成为瓶颈
2. linkis现有配置，功能，kyuubi是否支持、kyuubi提交任务时候占用的资源，归还资源的时机
3. 日志方案，启动日志
4. 状态方案、
5. 监控方案
6. kyuubi压测，极限收缩资源后是否支持
7. 脱敏加密查询
8. 适配层拉通海豚任务实例
9. 扩充中间件能力
10. 适配层平滑发版
11. 应用层配合修改代价、如果实现一个新的客户端代价，调度、解析、结果处理放到客户端

五、概要设计

6.1、组件划分



Calc-Cluster-API-Gateway

中间件对外的统一的入口服务，获取用户信息，转至UserControl做用户认证，或者校验token

Calc-Mid-Adapter

中间件对应用层的适配层，适配应用层调度、任务、即席查询请求，兼容原Linkis相关所有请求。对于任务类的请求，适配层负责完成任务持久化，任务状态追踪，**日志记录**等能力，应用层获取的任务相关的所有请求都由适配层完成，不依赖下游中间件服务。

UserControl

负责中间件租户、用户、资源、角色相关能力，并配合Ranger完成数据权限管控

Ranger

权限控制中心，借助开源Ranger结合Spark、Flink、Trino，完成引擎侧的权限Catalog、Database、Table、Column、Row 及加密脱敏管控

MetadataCenter

元数据中心，包含三个服务，**HiveMetaStore（容器化）**、**MetadataCollector**、**MetadataConnector**

HiveMetaStore

容器化的HiveMetaStore服务，原linkis侧自研，整合了原生的HiveMetaStore能力，同时增加元数据操作的各种Hook，为权限管控留下切入点；容器内的HiveMetaStore完全脱离Hadoop体系，在K8S内以微服务的形式对Spark、Flink、Trino提供元数据服务

MetadataCollector

元数据采集服务，为应用层元数据中心提供多元异构的元数据采集能力

MetadataConnector

提供应用层目前直连的方式获取元数据的能力

Kyuubi

Spark

Spark引擎，支持Spark Sql， Spark Scala

Flink/Dlink

Flink引擎，整合Dlink的能力，除了提供dlink cdcsource能力外，兼容flink原生SQL能力
后期需引入Ranger 插件，和权限中心结合起来

Trino/openlookeng

后期需引入Ranger 插件，和权限中心结合起来

JDBC: Mysql/StarRocks

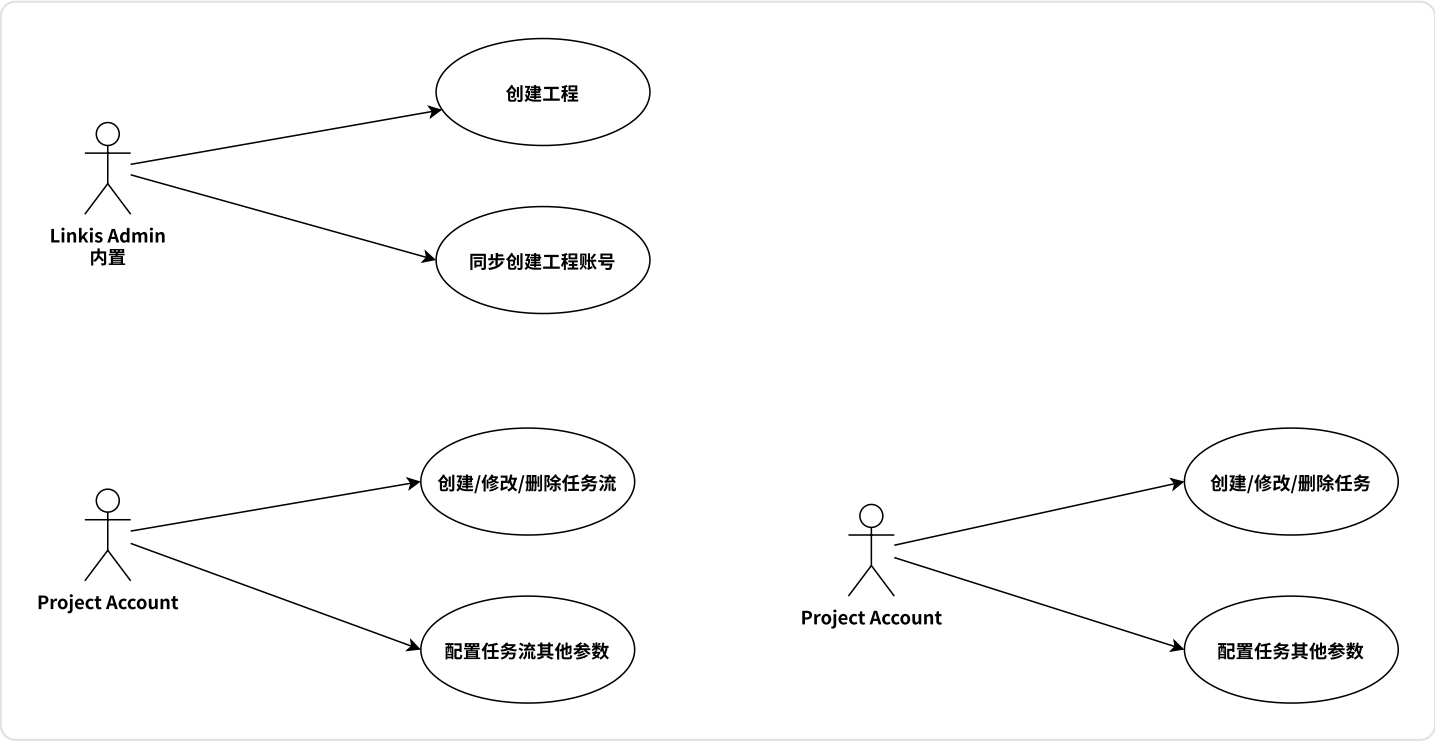
扩展支持StarRocks引擎

Linkis

原大数据中间件服务

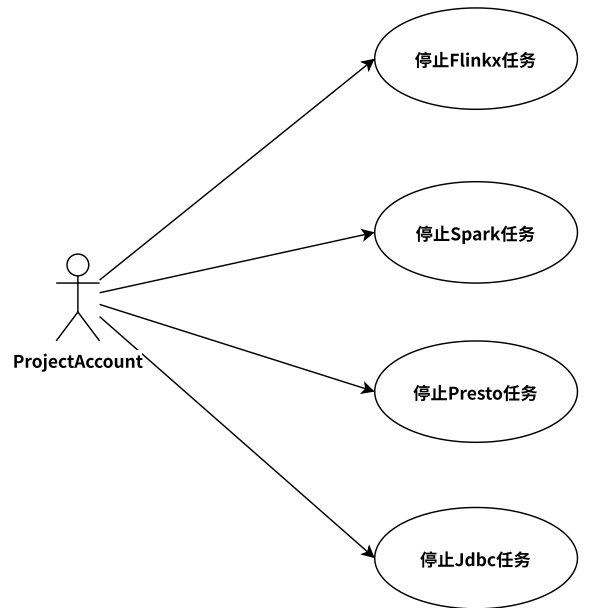
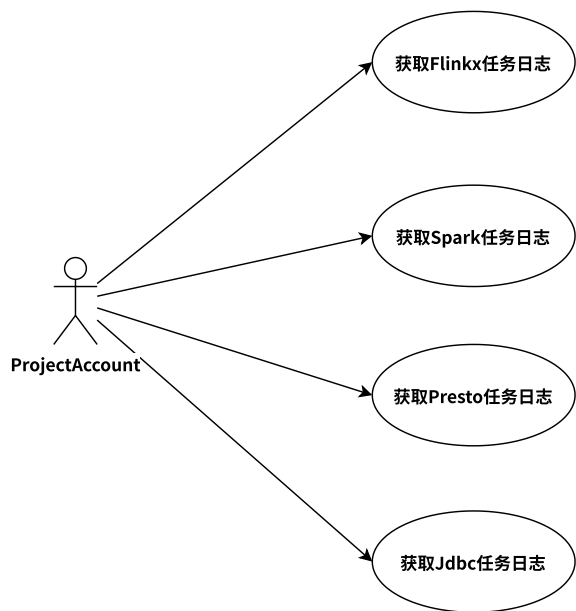
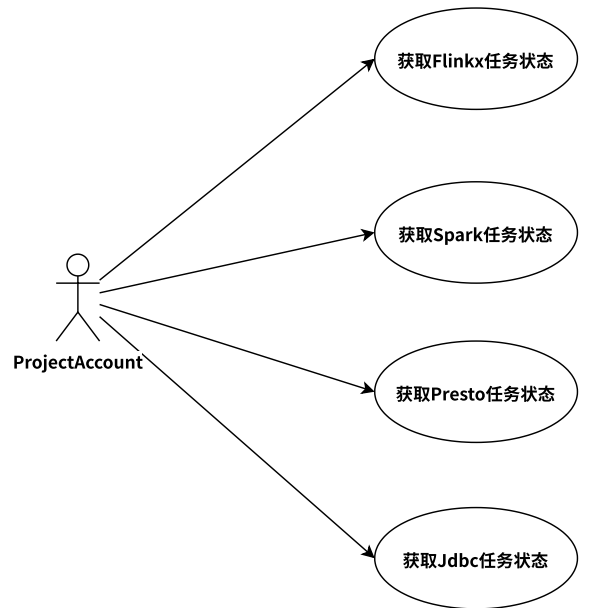
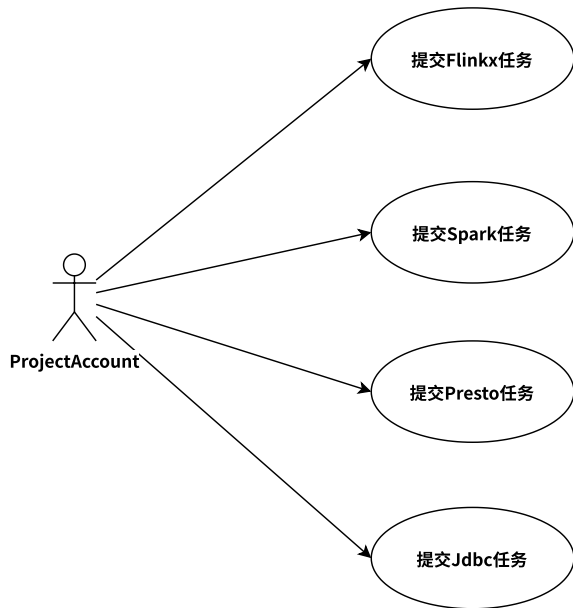
6.2、用例设计

Dolphin工程任务流任务相关



Spark、Flink、JDBC任务相关

- 应用层透传给Linkis的账户信息
- user：工程账号
- creator：项目管理平台userID

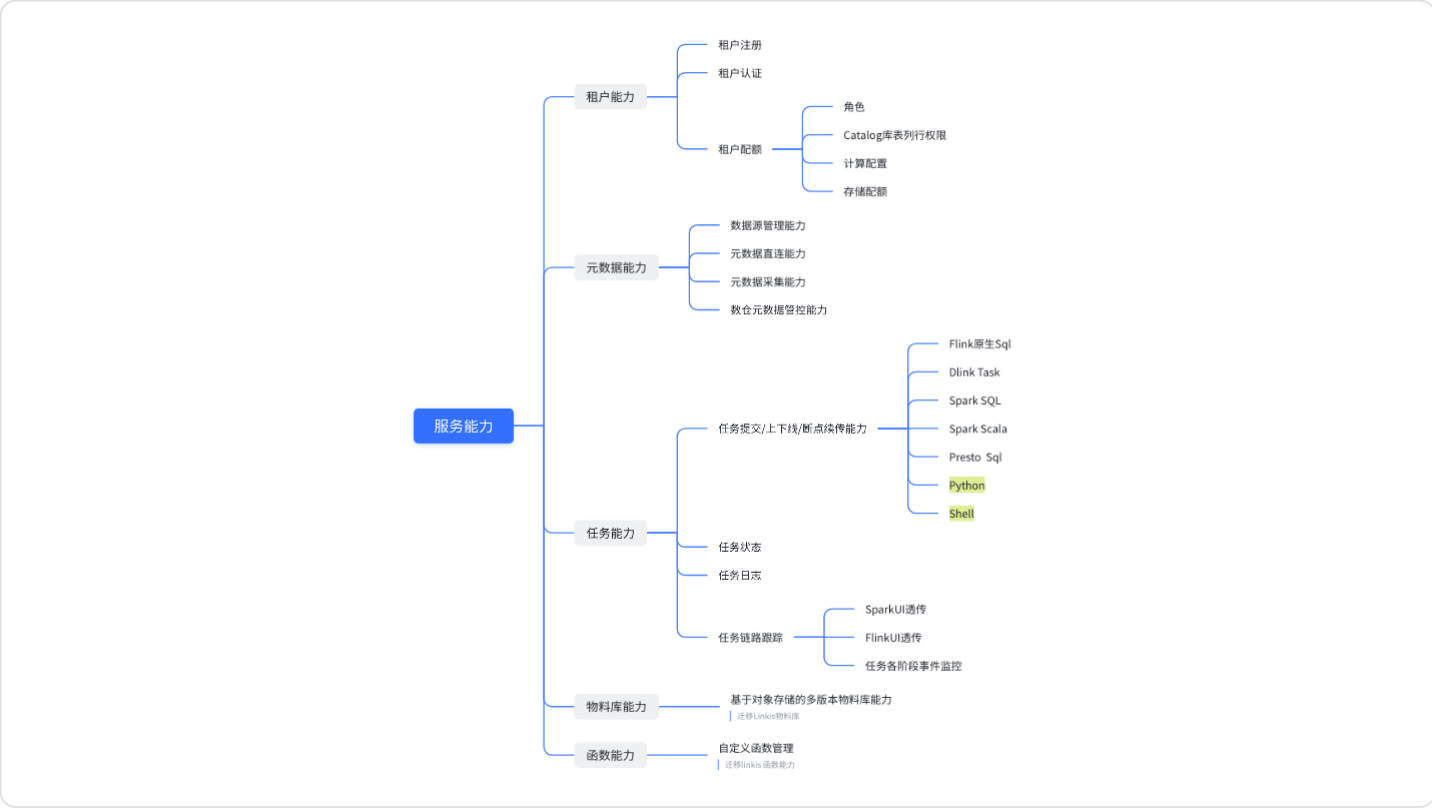


Presto即席查询相关

采集任务相关

6.3、能力梳理

服务能力



管理能力



6.4、 组件设计

6.4.1、 Calc-Cluster-Api-Gateway

6.4.2、 Calc-Mid-Adapter

6.4.3、 Calc-UserControl

6.4.4、 MetadataCenter

6.4.4.1 HiveMetaStore

6.4.4.2 MetadataConnection

6.4.4.3 MetadataConnector

6.4.5、转发

1、将calc gateway linkis从直接转发到linkis-gateway方式，修改为从calc gateway linkis 转发到适配层服务。

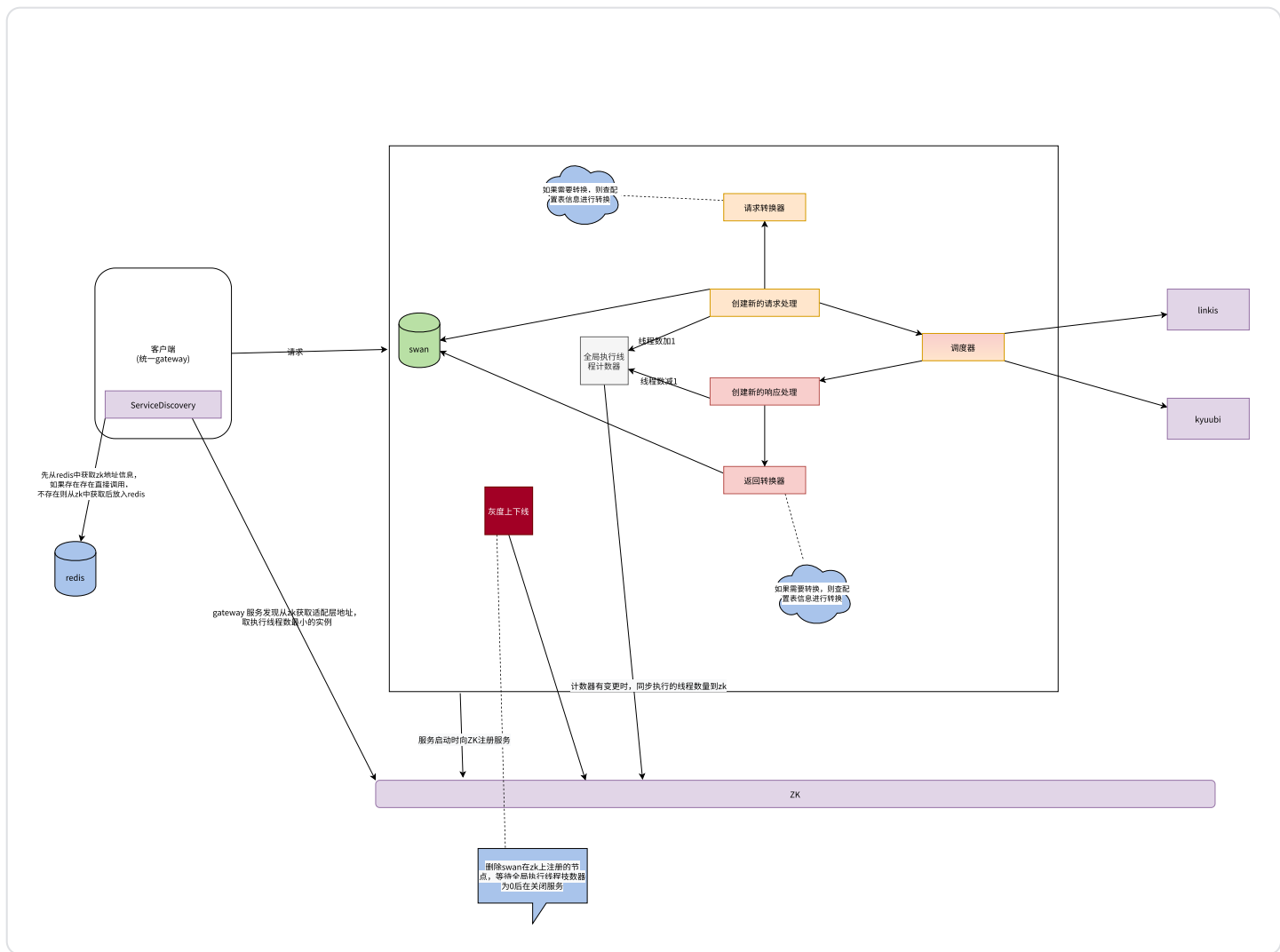
2、将接收到的请求，封装为各个服务需要的请求连接和参数，转发出去。

6.4.6、运营控制台

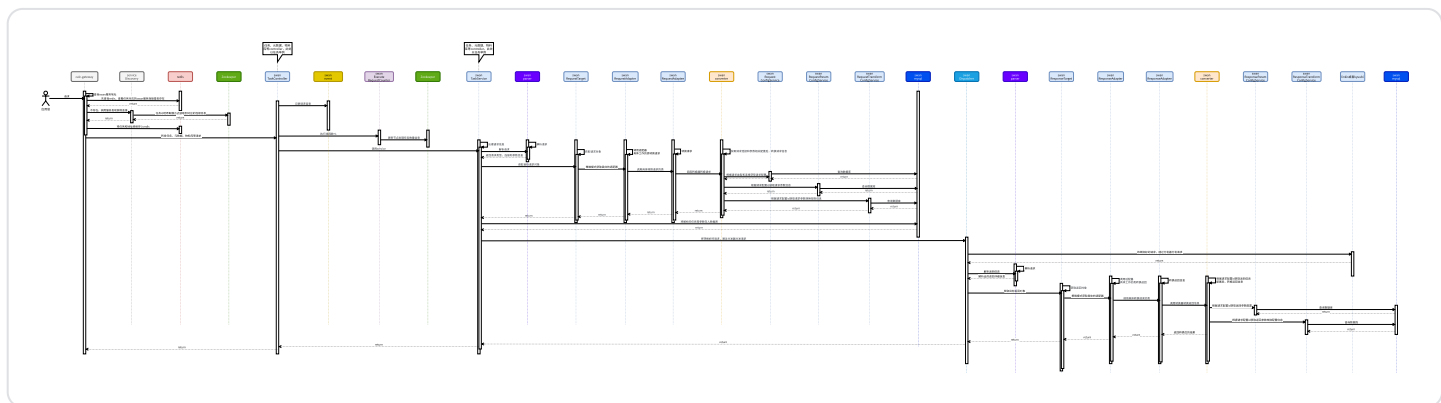
6.4.7、适配层方案设计

6.4.7.1、计算适配层与用户中心的方案设计





6.4.7.3、时序图



6.4.7.4、问题

1. 杀任务是否可以在适配层做
2. zk 支持的能力
3. 删除zk节点后, 后续节点会顶替, 导致hash路由混乱
4. 请求配置放到内存请求里
5. 按租户创建连接池, 缓存多个连接

是否需要swan服务启动时创建

6. spark sql 任务相关接口

7. batch kyuubi 、 session kyuubi 是否分开部署

跑实时任务走batch 模式

8. 两套适配层服务

9. 中间件服务发现方式:

a. 走 k8s service

i. gateway 、 usercenter 、 spark history、 metadata、 hivemetastore

b. 走zookeeper

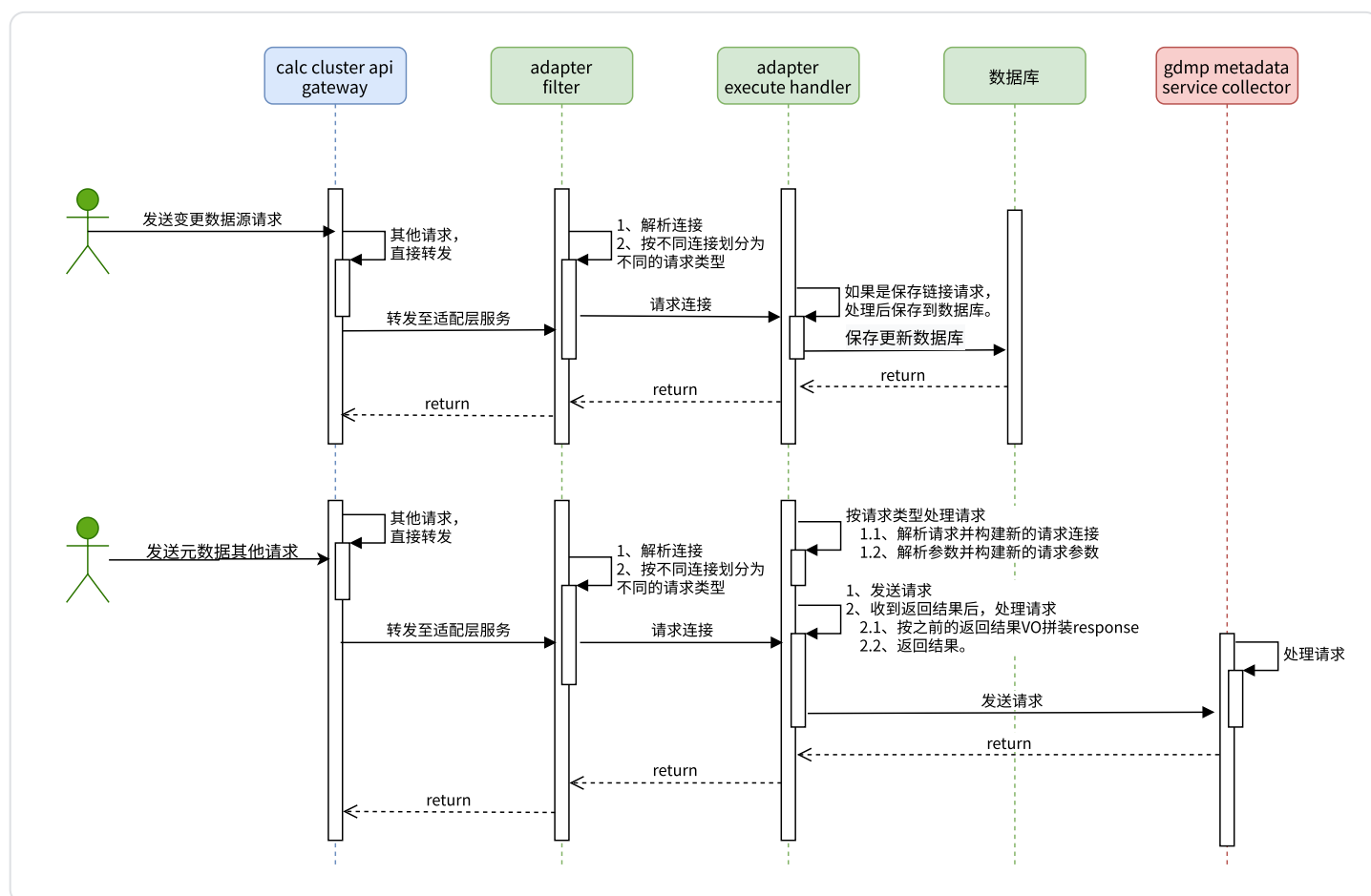
i. swan、 kyuubi

6.5、 任务拆解

6.5.1 领域模型设计

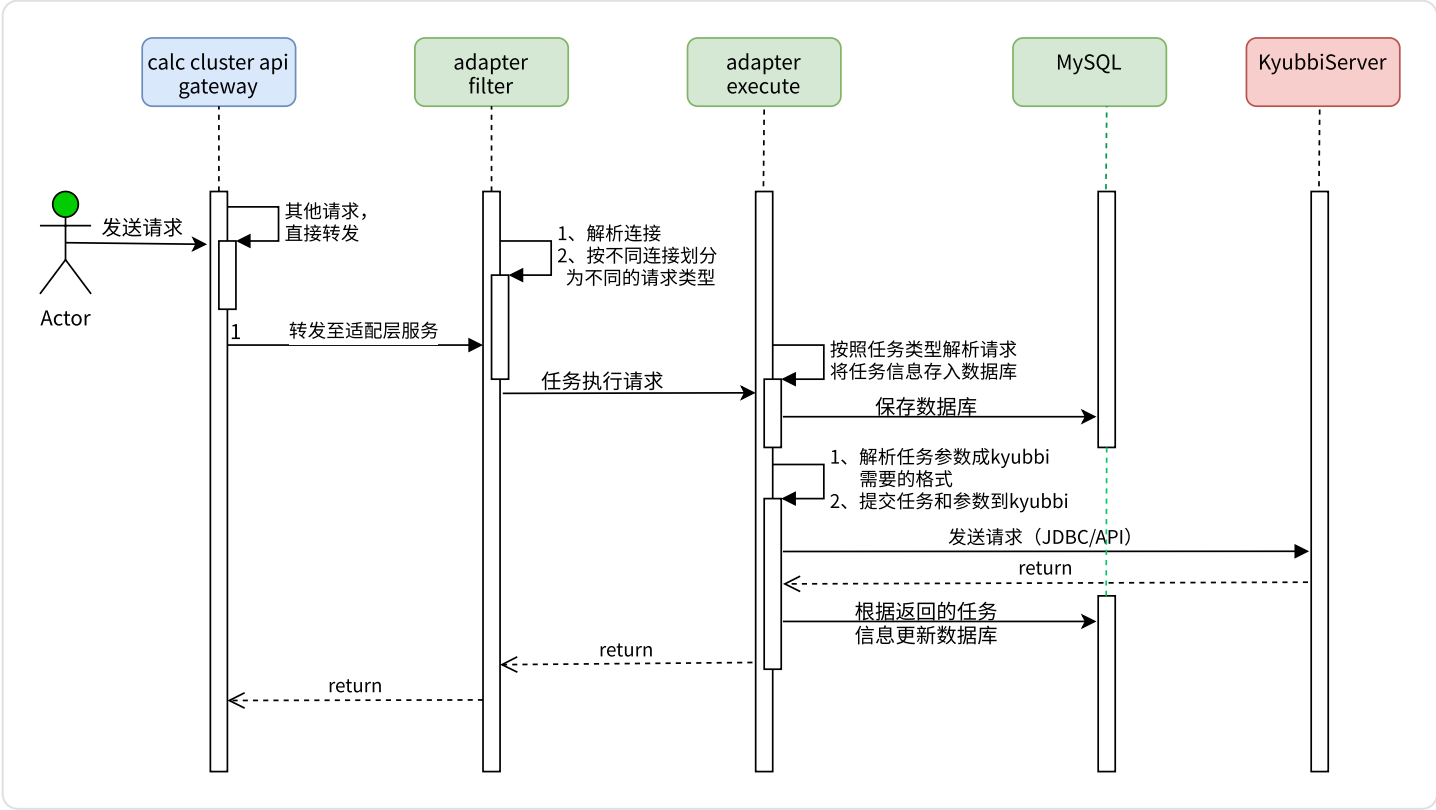
6.5.2 按接口分类画时序图(linkis 原接口)

6.5.2.1、 元数据相关

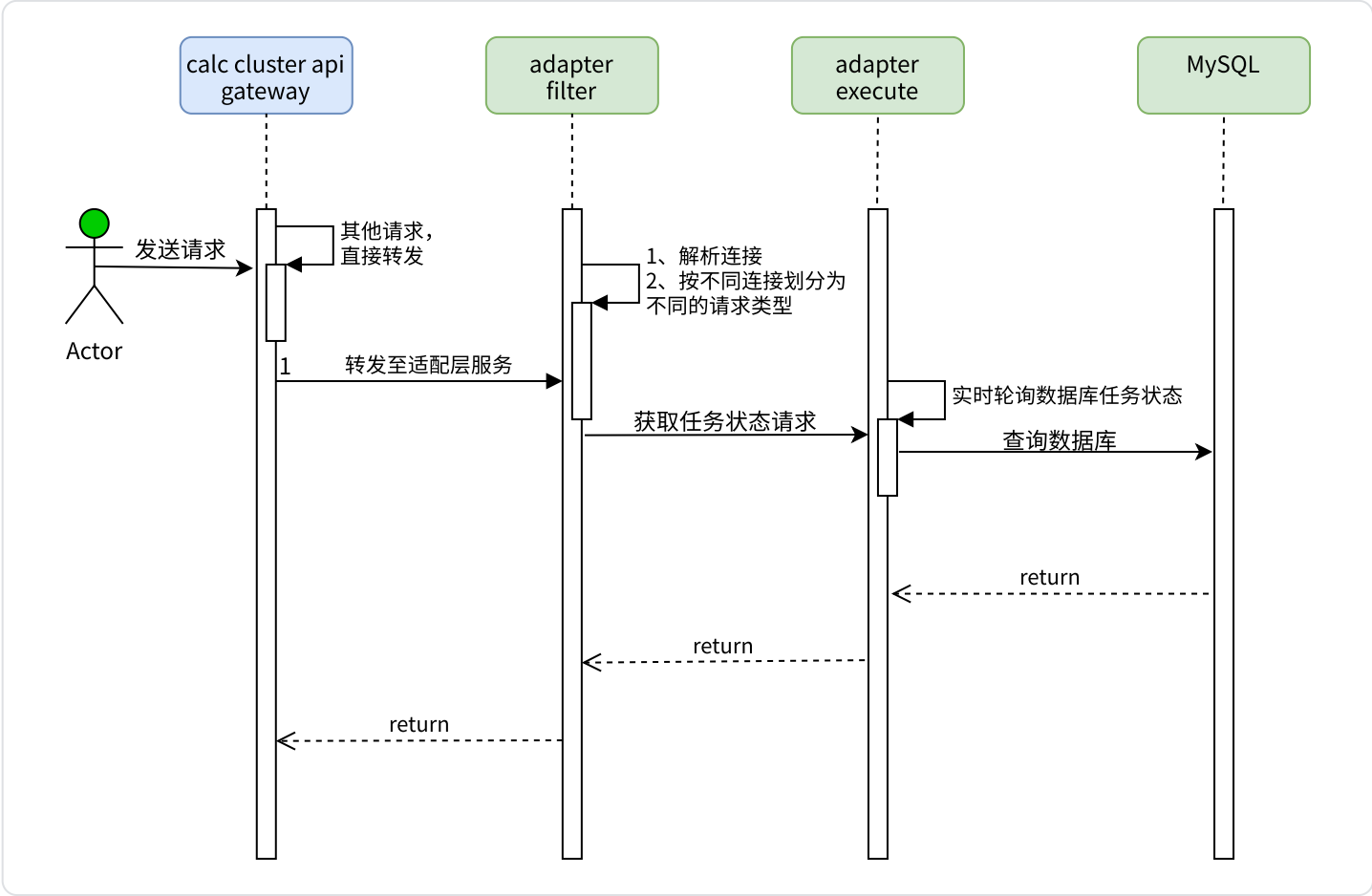


6.5.2.2、 任务相关

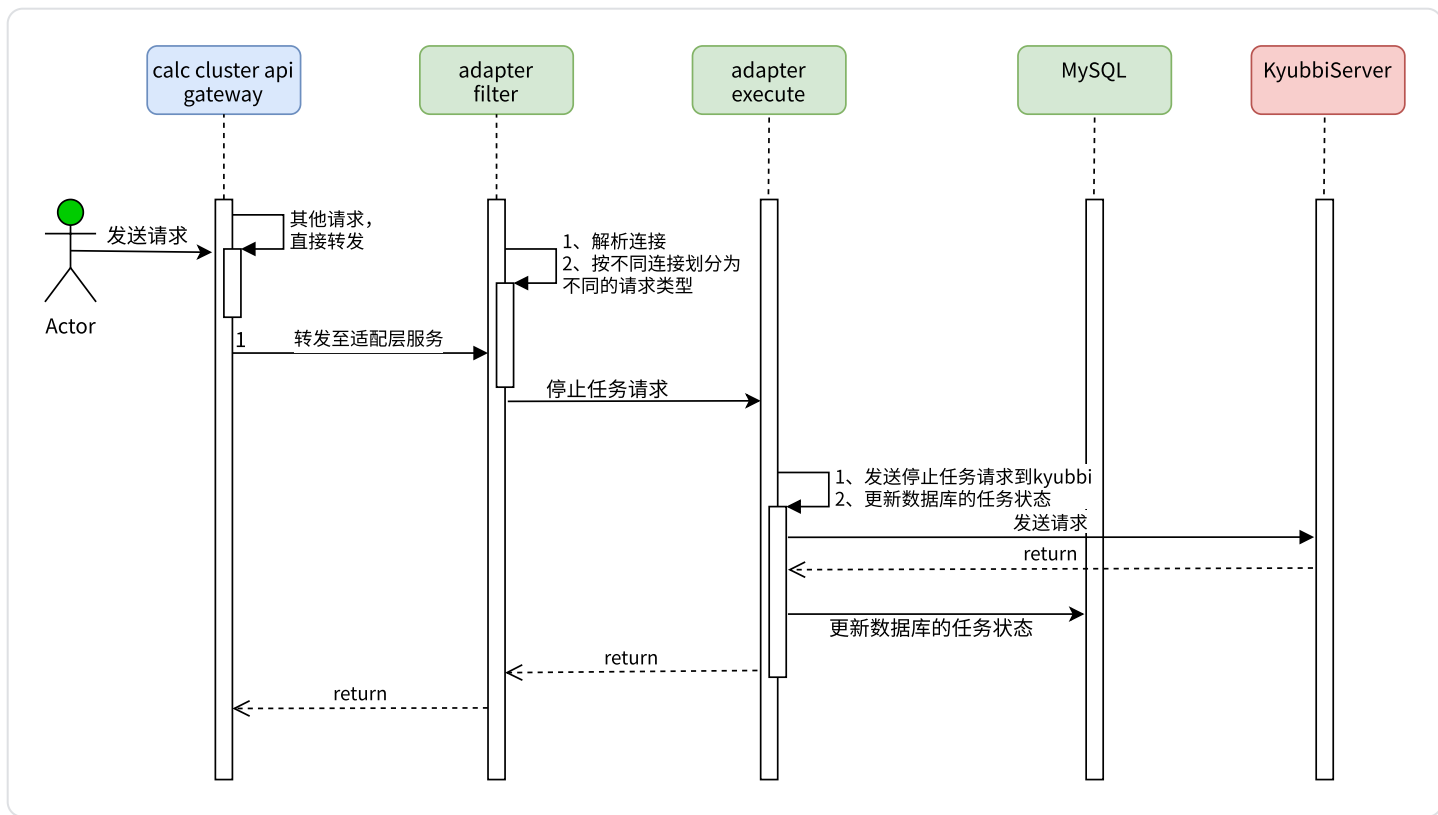
提交任务



任务状态

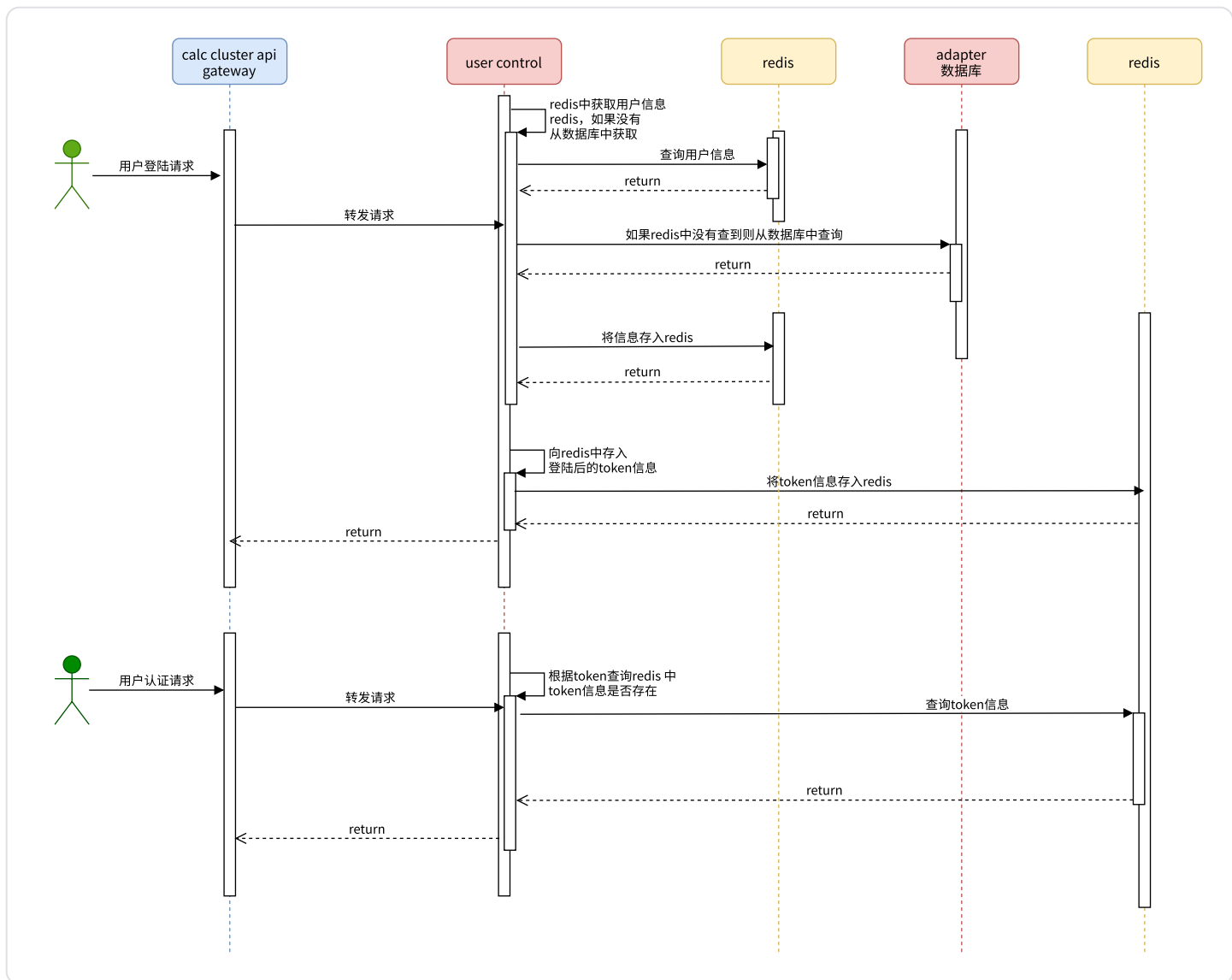


停止任务

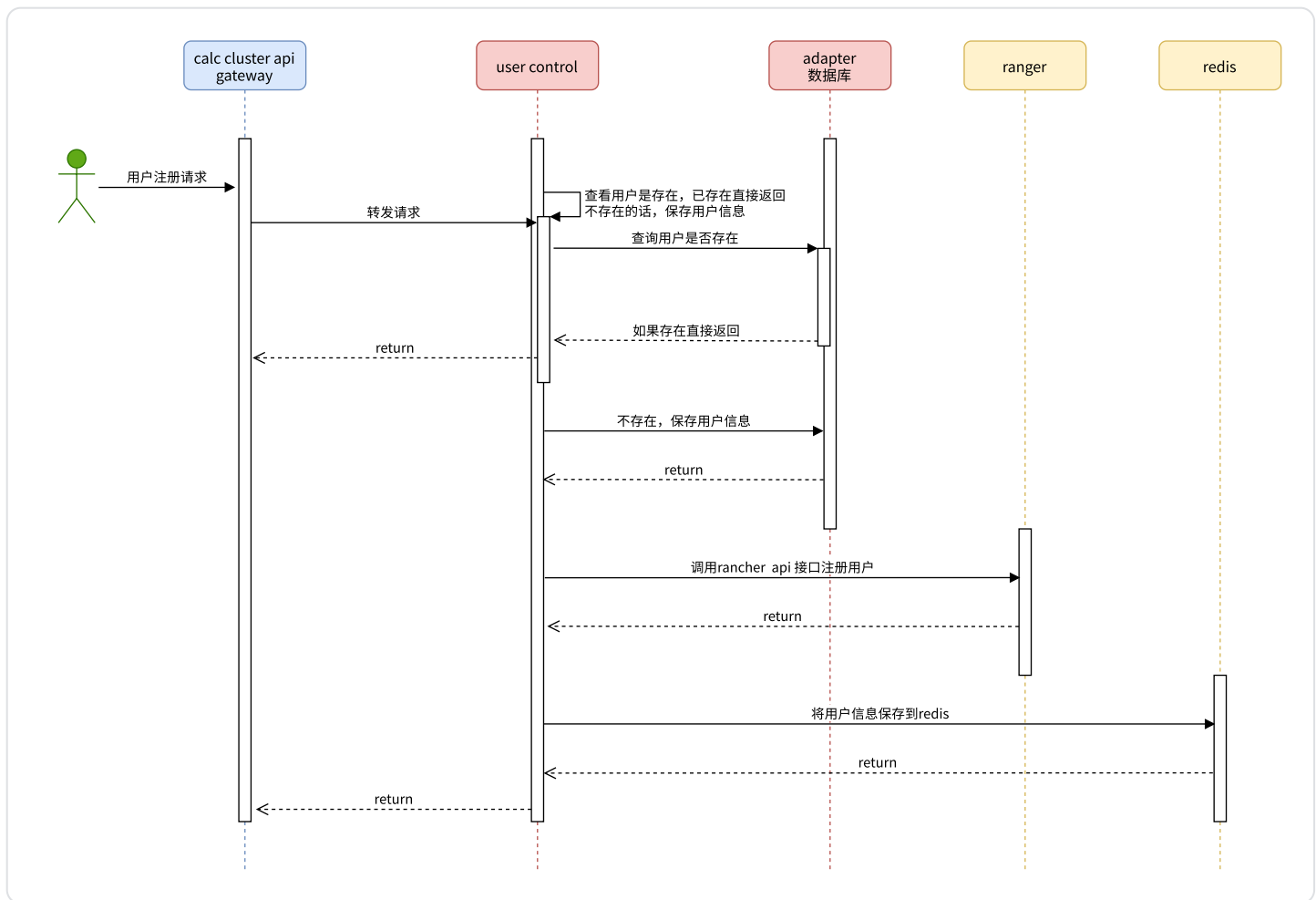


6.5.2.3、用户相关

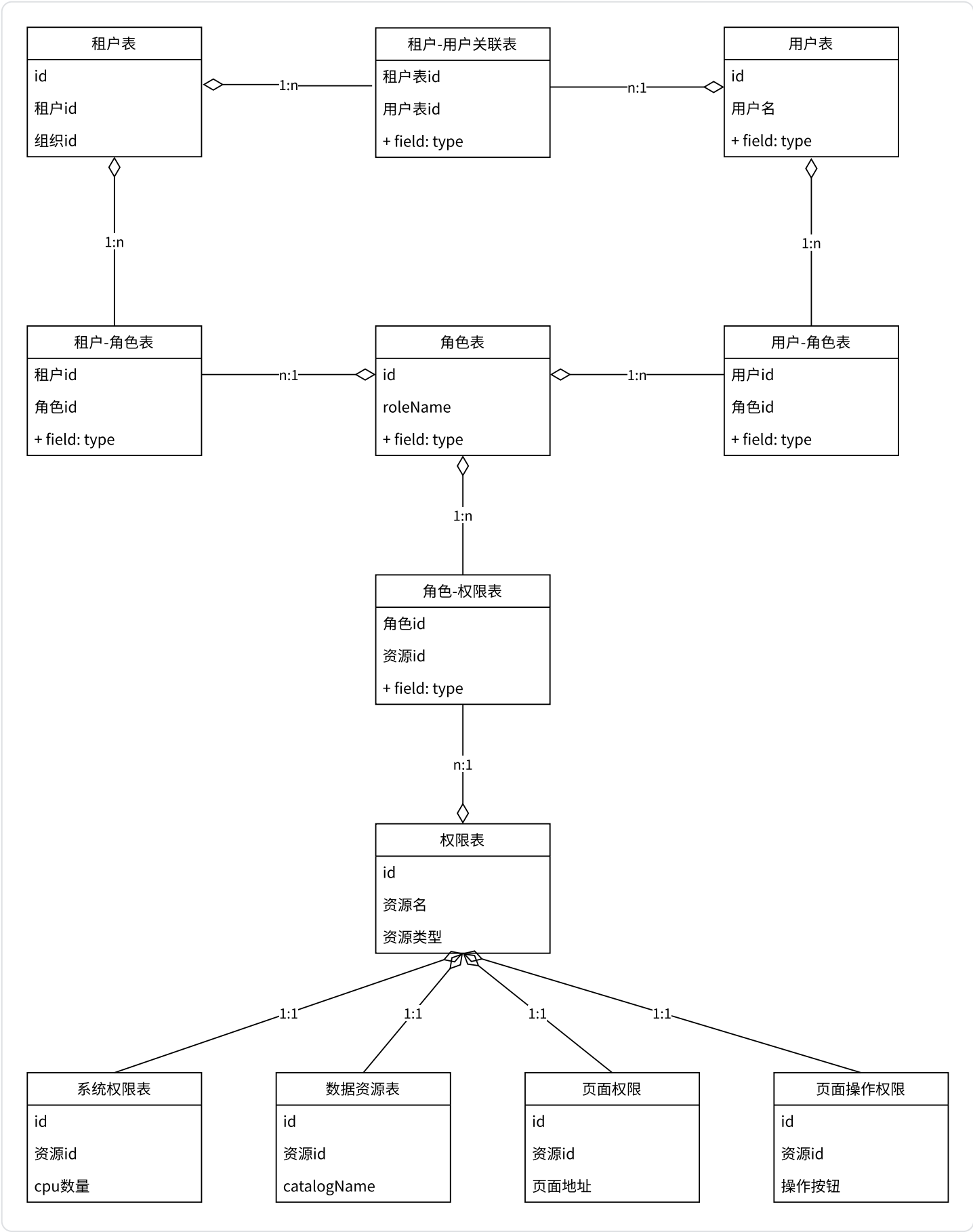
- 认证、登陆



- 注册



6.5.2.4、UC领域模型



6.5.2.5、物料库

- 上传
- 下载

- 查询
- 删除
- 获取所有版本

6.6、表结构



 swan.sql

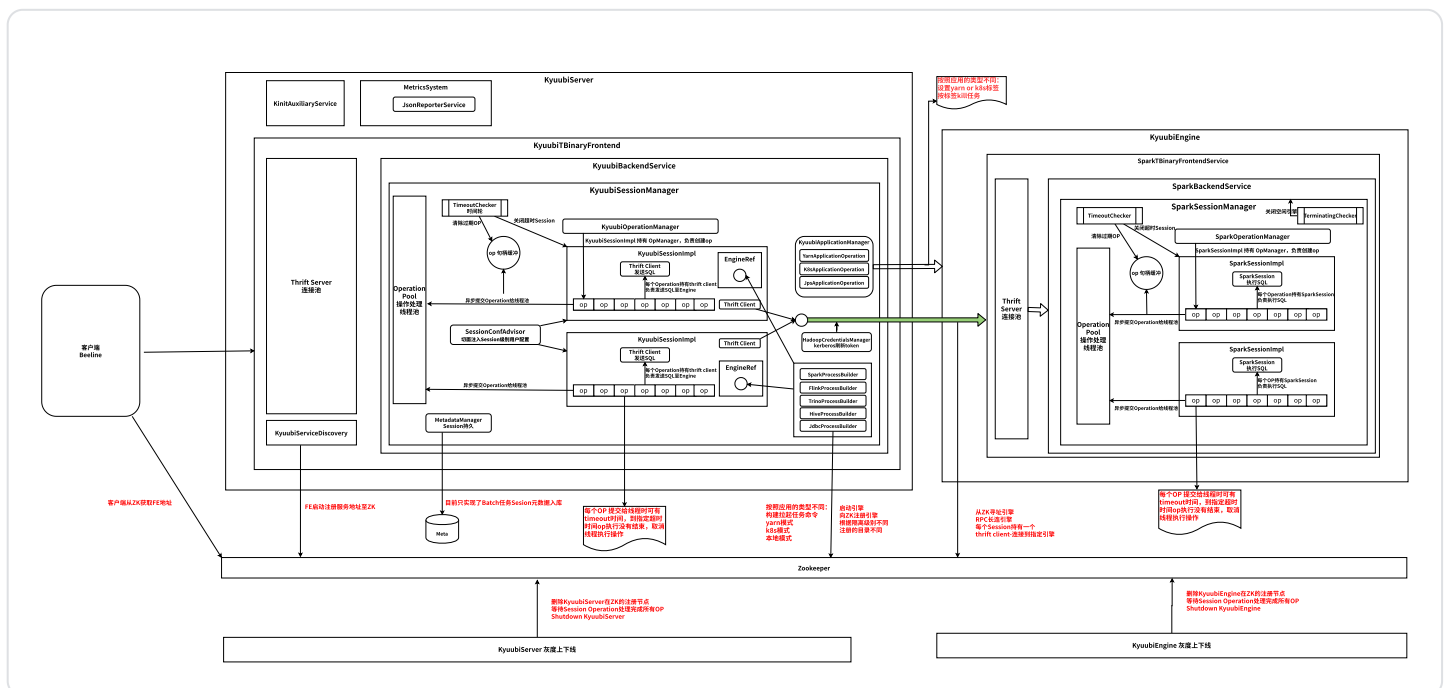
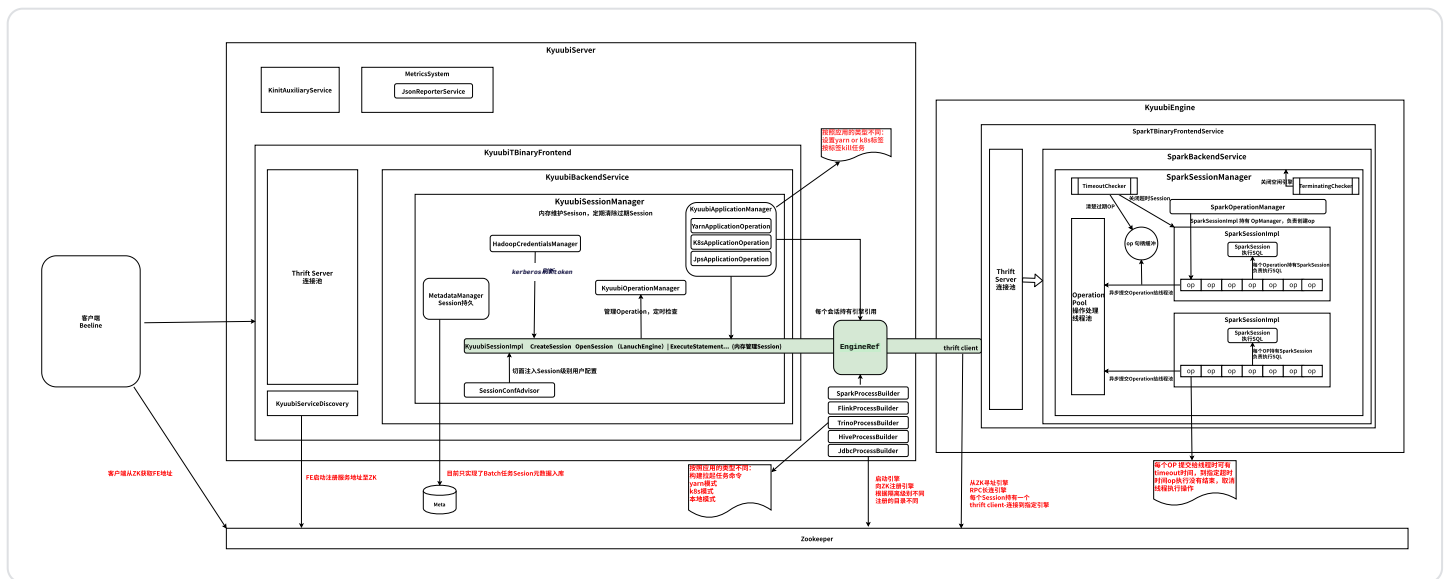
下边为 *PowerDesigner* 文件



swan.pdm
340.38KB

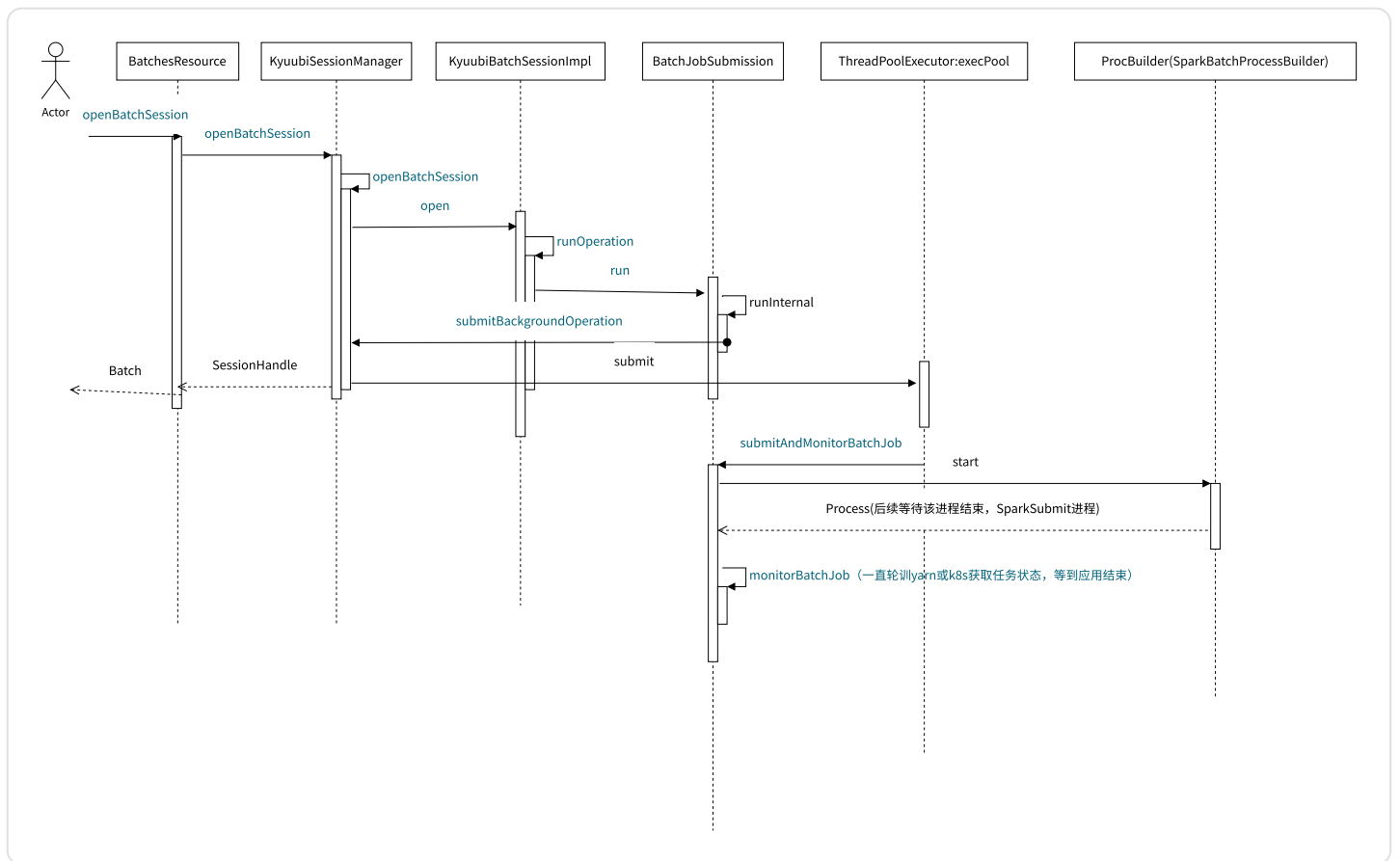


六、Kyuubi 中间件



Batch

1. 目前只支持Spark jar
2. 任务提交后，会占用线程池的一个线程。该线程负责启动任务，监控任务状态，等待任务结束
3. KyuubiServer重启后，会走 `org.apache.kyuubi.server.KyuubiRestFrontendService#recoverBatchSessions`，进行实例恢复，恢复时，依据kyuubiServer的地址过滤（在K8S中，KyuubiServer需要部署为 `StatefulSet?`）



适配层需要定期清理batch

batch任务完成或失败后，即使调用了delete，pod也不会被删除。是由适配层清理？还是修改org.apache.kyuubi.operation.BatchJobSubmission#close代码？【修改kyuubi代码】

linkis入口的interceptor能力在哪做？

```

CSEntranceInterceptor (com.webank.v
CommentInterceptor (com.webank.v
DBInfoCompleteInterceptor (com.v
LogPathCreateInterceptor (com.we
PythonCodeCheckInterceptor (com.
RuntimeInterceptor (com.webank.v
SQLCodeCheckInterceptor (com.web
SQLLimitEntranceInterceptor (com
ScalaCodeInterceptor (com.webank
SparkCodeCheckInterceptor (com.v
StorePathEntranceInterceptor (cc
VarSubstitutionInterceptor (com.
  
```

重点是：变量替换、SQL增加limit。

SQL增加limit

Linkis基于字符串替换的方法有bug，会导致SQL处理后变为错误的SQL。最好能基于解析器实现？

JDBC引擎问题

jdbc引擎目前的实现为在kyuubiServer的pod中启动一个java进程

改造点：

JDBC Trino Hive 等都是本地进程，需要改造弹出POD

FLINK引擎问题

Flink引擎内部借助flink sql client 负责把实时任务送上 yarn or K8s ，然后operation 返回即为成功。

此设计为了不让实时任务占用KyuubiServer 和 KyuubiEngine资源，是合理的，但是我们面临一个问题，实时任务在运行时环境的里状态是未知的。同时FlinkEngine 还未实现FlinkEvent埋点事件

改造点：

增加FlinkEvent 埋点，flinkEngine 把实时任务送上Yarn 或者 K8S之后，通过FlinkEvent事件将实时任务信息返回适配层，适配层通过 jobid， yarn 或者 k8s 信息，维护实时任务在运行时环境里的状态，此时和Kyuubi完全脱离，不占用Kyuubi任何资源。

Flink jar

走batch

Function问题

目前kyuubi对函数的支持还不完善，以Spark为例，在Session打开时，静态注册了`kyuubi_version()` ,`engine_name()` ,`engine_id()` ,`system_user()` ,`session_user()` 等系统函数。但是对用户级的函数支持目前没有找到切入点。例如，我们可以通过Spark sql 的 `add jar` , `create function` 的方式构建函数，但是依赖的jar 和 资源库的关系如何处理，这块需要结合适配层来整体考虑函数支持。

适配层统一维护资源和函数，资源文件存储在对象存储里，函数元数据存储在适配层的RDB里，适配层定时或者在资源发生变更时，将对象存储的资源下载到共享盘里，以SparkSession为例，KyuubiServer启动引擎后，在SparkSession 的sql里，`add jar` , `create function`

函数的形态：

- 1： 代码片段，根据租户的函数权限，在SparkSession 的Scala模式下，直接装载。
- 2： jar 方式：直接在SparkSession的 sql 里，`add jar` , `create function`

字段级血缘问题

标记血缘延续以前的实现，字段级血缘问题，参考点入下：

提取思路可以参考 kyuubi 血缘插件：<https://github.com/apache/incubator-kyuubi/blob/master/extensions/spark/kyuubi-spark-lineage/src/main/scala/org/apache/kyuubi/plugin/lineage/helper/SparkSQLLineageParseHelper.scala>

KyuubiServer插件扩展SessionConfAdvisor

SessionConfAdvisor 插件机制为各类租户的引擎提供了千人千面的conf切入点，给我们带来的收益点：

1. 结合适配层租户的资源配置，为租户动态设置不同的引擎配置，甚至是任务配置
2. 结合适配层的租户关联镜像管理，为租户提供多样的镜像挂载

Flinkx引擎扩展？

兼容以前任务，后逐步转移到原生Flink模式

kyuubiServer 拼接命令，利用flinkx 原生的命令行模式直接甩出，on yarn | on K8S

- 1：实时任务：类batch模式，单需要实现异步，实时任务提交完成，KyuubiServer 线程退出，扩展Flinkx程序，增加KyuubiServer 的日志和事件，状态能力，直接报送适配层。
- 2：离线任务：类batch模式，考虑并发给KyuubiServer 代理的压力，结合适配层的资源配置和KyuubiServer 的限流？

FlinkStreamSql引擎扩展？

兼容以前模式，后 逐步转移到原生Flink模式

类batch模式，kyuubiServer 拼接命令，利用flinkStreamSql 原生的命令行模式直接甩出，on yarn | on K8S

同Flinkx 引擎扩展的第二点

Dlink引擎扩展？

这个比较纠结，是不是需要和flink引擎合并还是和 FlinkxEngine 一样直接甩出去，建议，独立出来，这样Kyuubi原生的flinkEngine可以跟社区走（目前外部全部是on k8s 场景）

权限管控（优先考虑Spark）

1. 用户中心 会结合ranger 设置租户的权限，包含：
 - a. Catalog （数据源级）

- b. 库、表、列、行级
 - c. 列级别的加密、脱敏
2. 针对Catalog 权限，SparkEngine启动时，会根据租户的Catalog权限动态挂载多Catalog，达到在 spark sql 内部，可以轻松实现 多源异构的数据集成，数据开发，数据同步等
 3. 库、表、列权限，sparkEngine 会内嵌ranger-Authz 插件，和权限中心的策略同步
 4. 行级权限，sparkEngine 通过ranger-Authz从ranger 权限中心拉取策略，更改执行计划
 5. 列级别的加密、脱敏同上

Kyuubi事件

- 扩展事件Handle 输出到es
- 扩展事件，实现SQL 统计，异常收集，资源消耗，对应“日志分析”的需要点

1. KyuubiServerInfoEvent

```
1 * @param serverName the server name
2 * @param startTime the time when the server started
3 * @param eventTime the time when the event made
4 * @param state the server states
5 * @param serverIP the server ip
6 * @param serverConf the server config
7 * @param serverEnv the server environment
8 打点阶段：
9 1：服务启动时
10 2：服务停止时
```

2. KyuubiSessionEvent

```
1
2 * @param sessionId server session id
```

```

3 * @param clientVersion client version
4 * @param sessionType the session type
5 * @param sessionName if user not specify it, we use empty string instead
6 * @param user session user
7 * @param clientIP client ip address
8 * @param serverIP A unique Kyuubi server id, e.g. kyuubi server ip address
  and port,
9 *
    it is useful if has multi-instance Kyuubi Server
10 * @param conf session config
11 * @param startTime session create time
12 * @param remoteSessionId remote engine session id
13 * @param engineId engine id. For engine on yarn, it is applicationId.
14 * @param openedTime session opened time
15 * @param endTime session end time
16 * @param totalOperations how many queries and meta calls
17 * @param exception the session exception, such as the exception that occur
  when opening session
18 打点阶段:
19 1: 会话建立
20 2: 会话打开后端引擎
21 3: 会话关闭

```

3. KyuubiOperationEvent

```

1 * @param statementId the unique identifier of a single operation
2 * @param remoteId the unique identifier of a single operation at engine side
3 * @param
    the sql that you execute
4 * @param shouldRunAsync the flag indicating whether the query runs
  synchronously or not
5 * @param
6
  the current operation state
7
8 * @param eventTime the time when the event created & logged
9 * @param createTime the time for changing to the current operation state
10 * @param startTime the time the query start to time of this operation
11 * @param completeTime time time the query ends
12 * @param exception: caught exception if have
13 * @param sessionId the identifier of the parent session
14 * @param sessionUser the authenticated client user
15 打点阶段:
16 1: ExecuteStatement Operation 创建时
17 2: ExecuteStatement Operation 状态翻转时

```

4. EngineEvent (Spark)

```
1 * @param applicationId application id a.k.a, the unique id for engine
2 * @param applicationName the application name
3 * @param owner the application user
4 * @param shareLevel the share level for this engine
5 * @param connectionUrl the jdbc connection string
6 * @param master the master type, yarn, k8s, local etc.
7 * @param sparkVersion short version of spark distribution
8 * @param webUrl the tracking url of this engine
9 * @param startTime start time
10 * @param endTime end time
11 * @param state the engine state
12 * @param diagnostic caught exceptions if any
13 * @param settings collection of all configurations of spark and kyuubi
14 打点阶段:
15 1: 引擎初始化
16 2: 引擎启动
17 3: 引擎崩溃
```

5. SessionEvent (Spark)

```
1 * @param sessionId the identifier of a session
2 * @param engineId the engine id
3 * @param startTime Start time
4 * @param endTime End time
5 * @param ip Client IP address
6 * @param serverIp Kyuubi Server IP address
7 * @param totalOperations how many queries and meta calls
8 打点阶段:
9 1: 会话打开时
10 2: 会话关闭时
```

6. SparkOperationEvent

```
1 * @param statementId the unique identifier of a single operation
2 * @param statement the sql that you execute
3 * @param shouldRunAsync the flag indicating whether the query runs
   synchronously or not
4 * @param state the current operation state
5 * @param eventTime the time when the event created & logged
6 * @param createTime the time for changing to the current operation state
```

```

7 * @param startTime the time the query start to time of this operation
8 * @param completeTime time time the query ends
9 * @param exception: caught exception if have
10 * @param sessionId the identifier of the parent session
11 * @param sessionUser the authenticated client user
12 * @param executionId the query execution id of this operation
13 打点阶段:
14 1: ExecuteStatement Operation 创建时
15 2: ExecuteStatement Operation 状态翻转时

```

7. TrinoEngineEvent

```

1 connectionUrl: String,
2 startTime: Long,
3 endTime: Long,
4 state: ServiceState,
5 diagnostic: String,
6 settings: Map[String, String]

```

8. TrinoSessionEvent

```

1 sessionId: String,
2 username: String,
3 ip: String,
4 connectionUrl: String,
5 catalog: String,
6 startTime: Long,
7 var endTime: Long = -1L,
8 var totalOperations: Int = 0

```

9. TrinoOperationEvent

```

1 * @param statementId the unique identifier of a single operation
2 * @param statement the sql that you execute
3 * @param shouldRunAsync the flag indicating whether the query runs
  synchronously or not
4 * @param state the current operation state
5 * @param eventTime the time when the event created & logged
6 * @param createTime the time for changing to the current operation state
7 * @param startTime the time the query start to time of this operation
8 * @param completeTime time time the query ends

```

```
9 * @param exception: caught exception if have
10 * @param sessionId the identifier of the parent session
11 * @param sessionUser the authenticated client user
```

10. HiveEngineEvent

```
1 connectionUrl: String,
2 startTime: Long,
3 endTime: Long,
4 state: ServiceState,
5 diagnostic: String,
6 settings: Map[String, String]
```

11. HiveSessionEvent

```
1 * @param sessionId the identifier of a session
2 * @param engineId the engine id
3 * @param startTime Start time
4 * @param endTime End time
5 * @param ip Client IP address
6 * @param serverIp Kyuubi Server IP address
7 * @param totalOperations how many queries and meta calls
```

12. HiveOperationEvent

```
1 * @param statementId the unique identifier of a single operation
2 * @param statement the sql that you execute
3 * @param shouldRunAsync the flag indicating whether the query runs
  synchronously or not
4 * @param state the current operation state
5 * @param eventTime the time when the event created & logged
6 * @param createTime the time for changing to the current operation state
7 * @param startTime the time the query start to time of this operation
8 * @param completeTime time time the query ends
9 * @param exception: caught exception if have
10 * @param sessionId the identifier of the parent session
11 * @param sessionUser the authenticated client user
```

13. Flink 和 Jdbc 暂无事件

14. Kyuubi Batch 事件

```
1 CREATE TABLE metadata(  
2     key_id bigint PRIMARY KEY AUTO_INCREMENT COMMENT 'the auto increment key  
   id',  
3     identifier varchar(36) NOT NULL COMMENT 'the identifier id, which is an  
   UUID',  
4     session_type varchar(128) NOT NULL COMMENT 'the session type, SQL or  
   BATCH',  
5     real_user varchar(1024) NOT NULL COMMENT 'the real user',  
6     user_name varchar(1024) NOT NULL COMMENT 'the user name, might be a proxy  
   user',  
7     ip_address varchar(512) COMMENT 'the client ip address',  
8     kyuubi_instance varchar(1024) NOT NULL COMMENT 'the kyuubi instance that  
   creates this',  
9     state varchar(128) NOT NULL COMMENT 'the session state',  
10    resource varchar(1024) COMMENT 'the main resource',  
11    class_name varchar(1024) COMMENT 'the main class name',  
12    request_name varchar(1024) COMMENT 'the request name',  
13    request_conf mediumtext COMMENT 'the request config map',  
14    request_args mediumtext COMMENT 'the request arguments',  
15    create_time BIGINT NOT NULL COMMENT 'the metadata create time',  
16    engine_type varchar(1024) NOT NULL COMMENT 'the engine type',  
17    cluster_manager varchar(128) COMMENT 'the engine cluster manager',  
18    engine_id varchar(128) COMMENT 'the engine application id',  
19    engine_name mediumtext COMMENT 'the engine application name',  
20    engine_url varchar(1024) COMMENT 'the engine tracking url',  
21    engine_state varchar(128) COMMENT 'the engine application state',  
22    engine_error mediumtext COMMENT 'the engine application diagnose',  
23    end_time bigint COMMENT 'the metadata end time',  
24    peer_instance_closed boolean default '0' COMMENT 'closed by peer kyuubi  
   instance',  
25    INDEX kyuubi_instance_index(kyuubi_instance),  
26    UNIQUE INDEX unique_identifier_index(identifier),  
27    INDEX user_name_index(user_name),  
28    INDEX engine_type_index(engine_type)  
29 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

平滑发版

原生优雅停止实现

操作方式

<https://kyuubi.apache.org/docs/latest/tools/kyuubi-ctl.html#delete-server>

```
1 bin/kyuubi-ctl delete server --host XXX --port YYY
```

过程分析

1. Kyuubi-ctl的delete命令会把kyuubiserver在zk中的节点删除。代码位置
`org.apache.kyuubi.ctl.cmd.delete.DeleteCommand#doRun`
2. Kyuubiserver中会监控自身的zk节点的状态，收到NodeDeleted事件后，会调用优雅停止

```
1 // ServiceDiscovery启动时，创建zk节点，并监控节点
2 org.apache.kyuubi.ha.client.ServiceDiscovery#start
3
4 // 节点被删除时，调用优雅停止
5 org.apache.kyuubi.ha.client.zookeeper.ZookeeperDiscoveryClient.DeRegisterWatches
```

在K8S中的实现方案

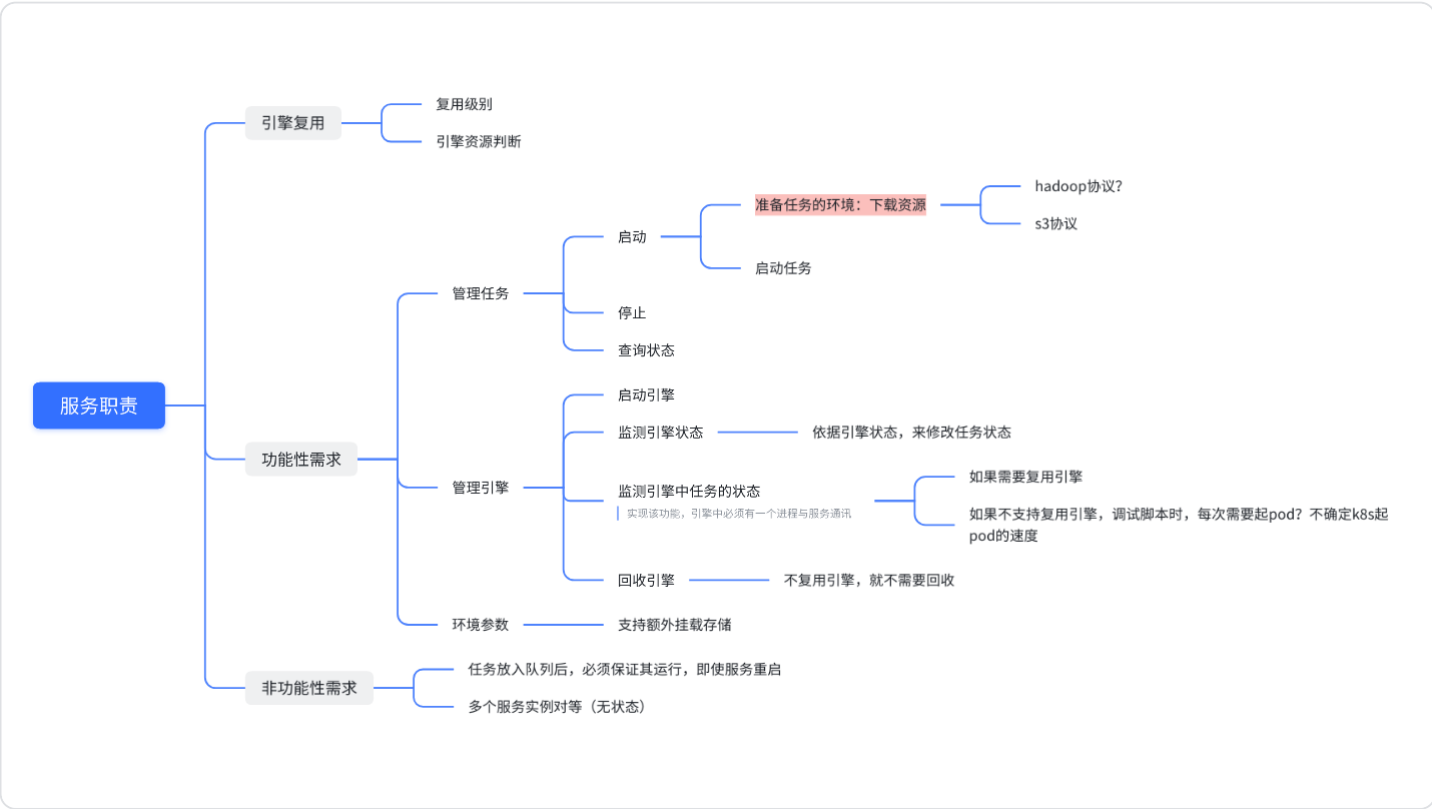
1. 设置pod的terminationGracePeriod参数，延长足够大
2. 在pod的preStop步骤中，调用kyuubi-ctl的delete命令

七、详细设计

执行Shell服务

需求分析

1. 支持shell、python类任务的运行、停止、状态查询
2. shell服务重启后，不影响任务（支持常驻任务）
3. 可以跟踪任务执行的整个流程
4. 引擎的复用？



详细设计

方案	资源占用分析	优点	缺点
方案1：独立的java服务			
方案2：用k8s的sparkoperator实现			
方案3：在kyuubi上增加shell的任务类型	任务模式 1个任务占用Kyuubi线程池的一个线程。jdk1.8 1个线程默认占用1M操作系统内存（参数可调（参考文章））32G能起，3.2w个线程		

方案1：独立的java服务

不支持复用引擎

方案3：在kyuubi上增加shell的任务类型

任务模式

优点：任务的模式和spark的batch模式是保持一致的，无需引擎，只需扩展一个operator即可，实现成本低。

缺点：k8s 弹出pod的时间消耗，如果快，是可以接受的，如果慢，用户不能接受。

耗时间：下载镜像、下载资源、启动任务

扩展BatchType的任务类型，支持K8S的pod

1. 实现org.apache.kyuubi.engine.ProcBuilder。该类用于生成yaml文件，进程是一个shell命令，命令内容为kubectl apply
2. ApplicationOperation用已实现的KubernetesApplicationOperation
3. 状态跟踪，停止任务，均使用BatchJobSubmission逻辑

这个和KyuubiServer operation 保持一致即可，Kyuubi会和Shell主进程的状态保持一致，但是无法感知Shell 内部的进程的状态（Kettle 本身的状态）

建议方案：

Shell基础镜像内部内置一个Java 自定义程序，负责两件事情：

- 1：监控Shell 内部Kettle的日志，从POD环境变量上获取SessionId， OperationId，报回ES，适配层能通过SessionId 和 OperationId 关联上启动日志
- 2：报送Event 埋点事件会 适配层的RDB，用于监控任务的关键节点

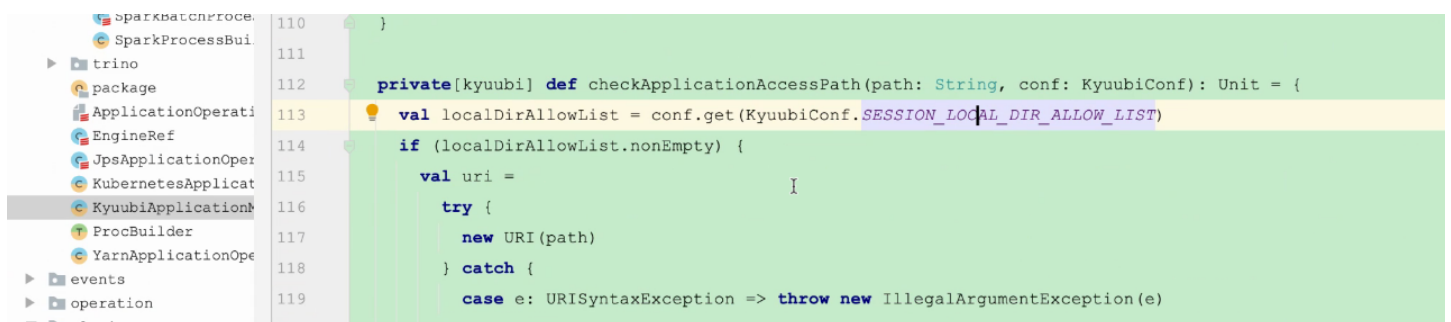
只监控shell主进程的状态和日志

4. 资源文件用initContainer进行挂载

这块的需求，需要统一考虑，不只是Shell的资源，还有Spark、Flink 函数的资源，KyuubiServer 目前给到一个统一的入口，就是可以根据conf 从KyuubiServer的本地目录加载。适配层的统一资源管理打算搬Linkis的，资源的存储肯定考虑对象存储，如果更改KyuubiServer 的资源加载从对象存储读取，效率是个问题；如果适配层的资源存储到KyuubiServer 的共享盘，数据可用性是个问题。

可以考虑两者结合起来，KyuubiServer 定时下载对象存储的资源？待讨论

看下kyuubiServer本地目录加载机制



统一的资源加载机制？适配层管理资源，放在对象存储中。（Spark UDF，shell的资源等）

5. 日志需要用额外的Container进行收集

方案1：kyuubiServer守护线程，拿日志。

方案2：用java实现shell引擎，通过shell引擎收集日志

- 1: KyuubiServer日志
- 日志统一考虑，KyuubiServer 日志统一输出到ES，目前kyuubi 按照 sessionId ， Operation 区分文件输出，考虑es日志增加 sessionId， Operation 的statementId ，和适配层结合起来
- 2: pod 内部日志
- 这个很久纠结？考虑shell的基础镜像内置日志采集能力，结合 SessionId 和 statementID 直接打入ES

调试模式（优先级低）

实现K8sPodOperation，kyuubi-k8spod-engine。engine的启动可由kubect执行。engine本身是一个java程序，注册zk等。

- 1：这个实现成本较高，如：扩展引擎的主进程实现，引擎侧的 FE ， BE， SessoinManager， SessionImp， 各类Operation， 肯定考兼容thrift 长连通讯
- 2：即使扩展完成，Engine自身环境会作为一个 Shell的运行环境。类似Flink 的engine，它自身只是一个client，负责将任务送上 yarn 或者 k8s，而Shell的运行环境，不能再POD。

待讨论问题

1. beeline接口与batch接口对比

接口类型		适合任务类型	
beeline接口	复用引擎 客户端保持连接	Sql， scala片段	
batch	客户端不用保持连接	Spark jar Flink jar	

任务模式走batch，调试模式走beeline？

kyuubi获取任务相关信息的方案

任务日志

- 方案一、通过kyuubi自带的org.apache.kyuubi.jdbc.KyuubiHiveDriver客户端获取
- 优点：是在获取日志的同时可以将应用层的任务信息都一并加入日志再写入到ES
- 缺点：1、任务量多时会造成客户端压力过大。2、客户端频繁调用也会造成kyuubiserver的压力过大

```

1 KyuubiStatement stmt = (KyuubiStatement) con.createStatement();
2 String queryId = stmt.getQueryId();
3 while (stmt.hasMoreLogs()) {
4     for (String line : stmt.getQueryLog(true, 1000)) {
5         System.out.println(line);
6         Thread.sleep(200);
7     }
8 }

```

方案二、在kyuubi-common的org.apache.kyuubi.operation.log.OperationLog这个类里直接将日志写入到kafka(选择方案二)

启动日志：

- 1、扩充OperationLog接口，直接写入kafka。
- 2、任务相关所有id都通过参数传递到OperationLog处，加入到日志里。
- 3、任务中心实例id，海豚任务流id，海豚任务实例id，适配层任务id，statementId，sessionId，
- 4、OperationLog写入Kafka时需要加一个自增id
- 5、Shell日志通过sidecar的filebeat采集
- 6、Shell实时任务占用kyuubiserver的一个线程

运行日志：

- 1、还是保持之前的log4j采集不变

待办：

验证断开连接查看任务状态是否能获取到（使用kyuubiHiveDriver）

结论：DriverManager.getConnection的连接不能断开，断开之后后续的任务都会close session.

优点：无需client循环调用，减少服务压力。

缺点：只能将sessionId和queryId传入到日志中，需要再次转换成应用层任务id关联任务日志。

```

1 def createOperationLog(session: Session, opHandle: OperationHandle):
  OperationLog = {
2   session.sessionManager.operationLogRoot.map { operationLogRoot =>
3     try {
4       val logPath = Paths.get(operationLogRoot,
  session.handle.identifier.toString)
5       val logFile = Paths.get(logPath.toAbsolutePath.toString,
  opHandle.identifier.toString)
6       info(s"Creating operation log file $logFile")
7       val queryId: String = session.getQueryId(opHandle)

```

```

8      val sessionId: UUID = session.handle.identifier
9      new OperationLog(logFile, queryId, sessionId)
10    } catch {
11      case e: IOException =>
12        error(s"Failed to create operation log for $opHandle in
13      ${session.handle}", e)
14      null
15    }.orNull
16  }

```

日志格式

```

1  {
2      "logLevel": "INFO",
3      "logFileName": "org.apache.kyuubi.operation.ExecuteStatement",
4      "statementId": "97f4f780-a8f5-4348-90e2-93db31881f5b",
5      "task_id": "1", //适配层任务id
6      "sessionId": "852b3755-e9a5-4305-8c12-702f3c864ed3",
7      "dolphins_process_instance_id": "34049", //海豚任务流实例id
8      "dolphins_task_instance_id": "34049", //海豚任务实例id
9      "collect_time": "2022-11-01 16:42:04.108",
10     "task_center_instance_id": "1", //任务中心实例id
11     "dolphins_process_id": "1", //海豚任务流id
12     "message": "2022-11-01 16:42:04.108 INFO
org.apache.kyuubi.operation.ExecuteStatement: Processing hive's query[97f4f780-
a8f5-4348-90e2-93db31881f5b]: RUNNING_STATE -> FINISHED_STATE, time taken:
0.041 seconds\r\n",
13     "sequenceId": 2, //自增id
14     "logTime": "2022-11-01 16:42:04.108"
15  }

```

任务参数传递

Kyuubi的jdbc url参数的定义

JDBC URL: `jdbc:hive2://<host>:<port>/dbName;sess_var_list?hive_conf_list#hive_var_list`
each list: `<key1>=<val1>;<key2>=<val2> and so on`

sess_var_list -> `sessConfMap`

hive_conf_list -> `hiveConfMap`

hive_var_list -> `hiveVarMap`

1、**sess_var_list** 相关的参数是在;之后

2、**hive_conf_list** 的参数是在? 之后

3、**hive_var_list** 参数是在#之后

4、扩展用户自定义taskconf的参数传递，可以在？或者#后面跟以swan开头的自定义参数，将会放入到taskconf里。

jdbc:hive2://10.1.125.40:10009/glodon_gdmp?

**swan.task.id=1;swan.task.center.instance.id=1;swan.dolphin.process.id=1;swan.dolphin.p
rocess.instance.id=1**

设置spark参数的格式以；连接

例：jdbc:hive2://localhost:10009/default;**spark.sql.shuffle.partitions=2;xxxxx=xxx**

Kyuubi的单个用户自定义的参数

```
___{username}___.{config key}
```

___kent___.spark.master=yarn

___kent___.spark.sql.adaptive.enabled=false

UDF

应用层上传时携带了工程对应的用户（swan_material表的owner）

应用层gdmp_base里面，t_base_linkis_user里记录了工程id及该工程对应的用户（目前工程和用户是一一对应的）

应用层udf的使用：

step1、上传jar包到物料库，返回version和resourceId

step2、调用udf的add接口填写resourceId和udfName，udf注册的语句等

step3、在使用的时候需要调用物料库下载接口将jar包下载到共享盘。（需要注意jar包路径，一个工程只需要下载一次就行）

note：产品设计需要考虑udf的共享，权限等问题。

一、主体流程：

应用层上传udf jar资源到物料库swan_material/version（传到obs），应用层新增udf到swan_udf表（新增时选择jar资源列表，把resourceId带过来，从而获取到jar资源在obs的路径）

1、swan里面下载udf jar到共享盘

<1> 这个下载是从swan接到execute请求时下载还是像flinkjar一样从kyuubi反调swan下载？

flinkjar一样统一一种处理方式，排队太久时token是否会失效

从kyuubi s3（sdk / hadoop api）下载（**可以直接指定--jars s3a://**）

<2> 工程关联问题：

目前kyuubi起引擎时默认的USER级别，用户是固定的linkis(配在swan里面的)，可以用工程对应的linkis用户，这样每个工程共用一个引擎(甚至未来可以每个工程共用n个引擎)，从而udf也是工程级别的共用

结论：目前阶段还是用固定linkis用户启动引擎，新增udf时把创建用户都手动写为linkis，根据linkis获取所有udf及其对应的jar(给北京建工先上去)

(spark.kyuubi.engine.share.level=USER --proxy-user linkis)

目前应用层上传资源jar包到物料库swan_material时，有资源id、文件名、工程用户等信息

2、kyuubi spark-submit启动引擎时 --jars传入udf jar包

从swan把对应任务的jar包所在共享盘路径传过来（起引擎时根据工程用户查询出该工程下所有udf jar资源）

验证--jars是否支持s3的路径，不支持的话add jar的方式（支持）

--jars 在kyuubi里面咋拼的

3、spark引擎启动时通过spark.sql(create temporary function xxx as 'com.xxx.xxx' using jar 'bml://xxxxxxx')

这步可以通过ENGINE_INITIALIZE_SQL、ENGINE_SESSION_INITIALIZE_SQL来做，kyuubi起完sparkSession有执行初始化sql机制（多个用分号分隔）

connnection模式是通过url传过来

batch(spark jar)是否需要udf功能(暂不考虑)

二、待议问题：

1、sparksql离线开发任务，创建一个任务时**kyuubi侧是引用该工程下的所有udf jar**，还是像flink jar一样每个任务单独引用资源文件的形式，目前应用层没有单个sparksql任务引用资源的功能

2、得让应用层调咱们**addUdf()**接口，目前这个和上传物料库是没有关联的，新增udf时表里加resourceID字段或者直接把jar包路径写到path(这块需要根据产品需求设计看下开放哪些udf相关的接口给应用层)

3、引擎复用策略，目前是user级别，在swan-server配置的为linkis，可以用**工程用户**替代

影响udf jar资源共用的问题

三、验证点：

一、验证spark 3.2.2从obs上拉取jar包：

1、把aws-java-sdk-bundle-1.11.375.jar、hadoop-aws-3.2.0.jar放到jars下

2、spark-defaults.conf下增加：

spark.hadoop.fs.s3a.impl=org.apache.hadoop.fs.s3a.S3AFileSystem

spark.hadoop.fs.s3a.endpoint=obs.cn-north-4.myhuaweicloud.com

spark.hadoop.fs.s3a.secret.key=aaaa

spark.hadoop.fs.s3a.access.key=bbbb

spark.hadoop.fs.s3a.aws.credentials.provider=org.apache.hadoop.fs.s3a.SimpleAWSCredentials
Provider

3、验证命令：

```
/home/linkis/app/spark-3.2.2-bin-hadoop3.2/bin/spark-submit \  
--class org.apache.spark.examples.SparkPi \  
--master local[1] \  
s3a://gdmp-linkis/sxren/spark-examples_2.12-3.2.2.jar \  
100
```

```
/home/linkis/app/spark-3.2.2-bin-hadoop3.2/bin/spark-submit \  
--class org.apache.spark.examples.SparkPi \  
--master local[1] \  
--jars s3a://gdmp-linkis/sxren/spark-examples_2.12-3.2.2.jar \  
s3a://gdmp-linkis/sxren/spark-examples_2.12-3.2.2.jar \  
100
```

```
/home/linkis/app/spark-3.2.2-bin-hadoop3.2/bin/spark-submit \  
--class org.apache.spark.examples.SparkPi \  
--master local[1] \  
--conf spark.jars=s3a://gdmp-linkis/sxren/spark-examples_2.12-3.2.2_test.jar \  
/home/linkis/app/spark-3.2.2-bin-hadoop3.2/examples/jars/spark-examples_2.12-3.2.2.jar \  
100
```

二、kyuubi是如何拼--jars的

kyuubi没有拼--jars是，我们可以通过--conf拼

--conf spark.jars=s3a://gdmp-linkis/sxren/spark-examples_2.12-3.2.2_test.jar 多个jar逗号分隔
kyuubi代码里：

所以我们可以拼在connUrl里：

```
bin/beeline -n linkis -u "jdbc:hive2://10.2.82.132:10009?
```

```
kyuubi.engine.type=SPARK_SQL;spark.executor.instances=1;spark.executor.cores=1;spark.executor.memory=512M;spark.submit.deployMode=cluster;spark.jars=s3a://gdmp-linkis/sxren/spark-examples_2.12-3.2.2_test.jar"
```

另外kyuubi还支持SESSION_LOCAL_DIR_ALLOW_LIST (kyuubi.session.local.dir.allow.list) 属性，用于设置允许kyuubi从哪些目录下载jar包，如果不设置表示没有限制

三、ENGINE_INITIALIZE_SQL、ENGINE_SESSION_INITIALIZE_SQL初始化参数

这2个参数还不能直接用，在引擎侧，他是从kyuubiConf里取的

需要加个参数，从swan里面拼进去，在引擎侧取出来，类似之前加scala 静默执行的那个，但是这里没有sessionConf了，需要从sparkConf里取出（所以就不能在共用一个引擎时，不同session打过来重复注册函数了）

```
beeline -n linkis -u "jdbc:hive2://10.1.131.29:10009?
```

```
kyuubi.engine.type=SPARK_SQL;spark.submit.deployMode=client;spark.udf.init.sql=aaa,bbb"
```

研究下能否把参数从url传到引擎侧的kyuubiConf：

可以，在openSession创建session的时候把sessionConf放到kyuubiConf里

四、sparkSession个数问题

多个，继承的关系 SparkSession.newSession()

Start a new session with isolated SQL configurations, temporary tables, registered functions are isolated, but sharing the underlying `SparkContext` and cached data.

测试udf jar及udf函数用户隔离情况

```
./beeline -u "jdbc:hive2://10.1.131.29:10009/;abc=cde?
```

```
kyuubi.engine.share.level=SERVER;swan.task.id=1;spark.executor.instances=1;spark.executor.cores=2;spark.sql.catalog.mysql_linkis_dev_root=org.apache.spark.sql.execution.datasources.v2.jdbc.JDBCTableCatalog;spark.sql.catalog.mysql_linkis_dev_root.url=jdbc:mysql://10.0.204.20:3306/linkis_debug;spark.sql.catalog.mysql_linkis_dev_root.user=root;spark.sql.catalog.mysql_linkis_dev_root.password=123456" -n aaa
```

```
./beeline -u "jdbc:hive2://10.1.131.29:10009/;?
```

```
kyuubi.engine.share.level=SERVER;swan.task.id=1;spark.executor.instances=1;spark.executor.co  
res=2;spark.sql.catalog.mysql_linkis_dev_root=org.apache.spark.sql.execution.datasources.v2.j  
dbc.JDBCTableCatalog;spark.sql.catalog.mysql_linkis_dev_root.url=jdbc:mysql://10.0.204.20:33  
06/dbz;spark.sql.catalog.mysql_linkis_dev_root.user=root;spark.sql.catalog.mysql_linkis_dev_r  
oot.dbtable=linkis_debug;spark.sql.catalog.mysql_linkis_dev_root.password=123456" -n bbb
```

udf注册

方式一、

-- 加载jar包

```
ADD jar hdfs:///tmp/spark-sql-udf-1.0-SNAPSHOT.jar;
```

-- 创建临时函数

```
CREATE TEMPORARY FUNCTION extractReaderUserName AS  
'com.liurx.demo.spark.sql.business.ExtractFlinkxUserNameUDF';
```

方式二、

```
create temporary function test_fun as
```

```
'com.liurx.demo.spark.sql.business.ExtractFlinkxUserNameUDF' using jar 'hdfs:///tmp/spark-  
sql-udf-1.0-SNAPSHOT.jar';
```

使用udf函数：

```
select test_fun(execution_code) from mysql_linkis_dev_root.linkis_dev.linkis_task;
```

结论：

2种方式都是jar没有用户隔离，临时函数有用户隔离

使用默认的USER共享级别时，openSession会创建新的sparkSession(用隔离的sql配置，产生的临时表、注册的udf函数都是隔离的，但是共享/基于相同的sparkContext及缓存数据)

血缘预研

一、kyuubi lineage实现（master分支）

1、介绍：

/Users/sx_ren/Documents/广联

达/source_code/kyuubi/docs/extensions/engines/spark/lineage.md

主要是通过扩展**QueryExecutionListener** <- **SparkOperationLineageQueryExecutionListener**
sql执行完成后会触发**SparkListenerSQLExecutionEnd**事件并被 **QueryExecutionListener**捕获，
并把解析的血缘信息以json的格式写入日志文件(中间其实是写到spark的事件总线 or kyuubi的事件总线)

```
1 ## table
2 create table test_table0(a string, b string)
3 ## query
4 select a as col0, b as col1 from test_table0
```

上面的sql产生的血缘如下（Lineage类，分别是输入表、输出表、列血缘）：

```
{
  "inputTables": ["default.test_table0"],
  "outputTables": [],
  "columnLineage": [{
    "column": "col0",
    "originalColumns": ["default.test_table0.a"]
  }, {
    "column": "col1",
    "originalColumns": ["default.test_table0.b"]
  }]
}
```

sql类型支持：spark's Command and Query type：

Query

- Select

Command

- AlterViewAsCommand
- AppendData
- CreateDataSourceTableAsSelectCommand
- CreateHiveTableAsSelectCommand

- CreateTableAsSelect
- CreateViewCommand
- InsertIntoDataSourceCommand
- InsertIntoDataSourceDirCommand
- InsertIntoHadoopFsRelationCommand
- InsertIntoHiveDirCommand
- InsertIntoHiveTable
- MergeIntoTable
- OptimizedCreateHiveTableAsSelectCommand
- OverwriteByExpression
- OverwritePartitionsDynamic
- ReplaceTableAsSelect
- SaveIntoDataSourceCommand

2、构建:

build/mvn clean package -pl :kyuubi-spark-lineage_2.12 -DskipTests

- The main plugin jar, which is under `./extensions/spark/kyuubi-spark-lineage/target/kyuubi-spark-lineage_${scala.binary.version}-${project.version}.jar`
- The least transitive dependencies needed, which are under `./extensions/spark/kyuubi-spark-lineage/target/scala-${scala.binary.version}/jars`

build/mvn clean package -pl :kyuubi-spark-lineage_2.12 -DskipTests -Dspark.version=3.1.2

支持的spark版本:

Spark Version	Supported	Remark
master	√	-
3.3.x	√	-
3.2.x	√	-
3.1.x	√	-
3.0.x	√	-
2.4.x	x	-

3、安装使用

把kyuubi-spark-lineage_*.jar放到spark_home/jars下

设置sparkConf:

spark.sql.queryExecutionListeners=org.apache.kyuubi.plugin.lineage.SparkOperationLineageQueryExecutionListener

4、获取血缘（复用引擎时与任务实例的关联-）

SparkOperationLineageQueryExecutionListener捕获事件并处理（处理过程就是解析逻辑执行计划）成血缘json后，写到了spark事件总线 或 kyuubi事件总线

spark.kyuubi.plugin.lineage.dispatchers: SPARK_EVENT(默认)、KYUUBI_EVENT

从spark事件总线获取血缘:

```
1 spark.sparkContext.addSparkListener(new SparkListener {
2     override def onOtherEvent(event: SparkListenerEvent): Unit = {
3         event match {
4             case lineageEvent: OperationLineageEvent =>
5                 // Your processing logic
6             case _ =>
7                 }
8         }
9     })
```

从kyuubi事件总线获取血缘: Kyuubi Event Handler

5、其他

测试类里有很多例子:

OperationLineageEventSuite、SparkSQLLineageParserHelperSuite

核心血缘解析逻辑执行计划的实现在LineageParser.parse()里，他考虑了带catalog前缀的情况

二、gudusoft商业化产品叫SqlFlow(官网: <https://www.gudusoft.com/>)

blog.csdn.net

gudusoft.gsqlparser-2.5.1.9.jar这个jar包还是可以用的

在线使用体验: <https://sqlflow.gudusoft.com/#/>

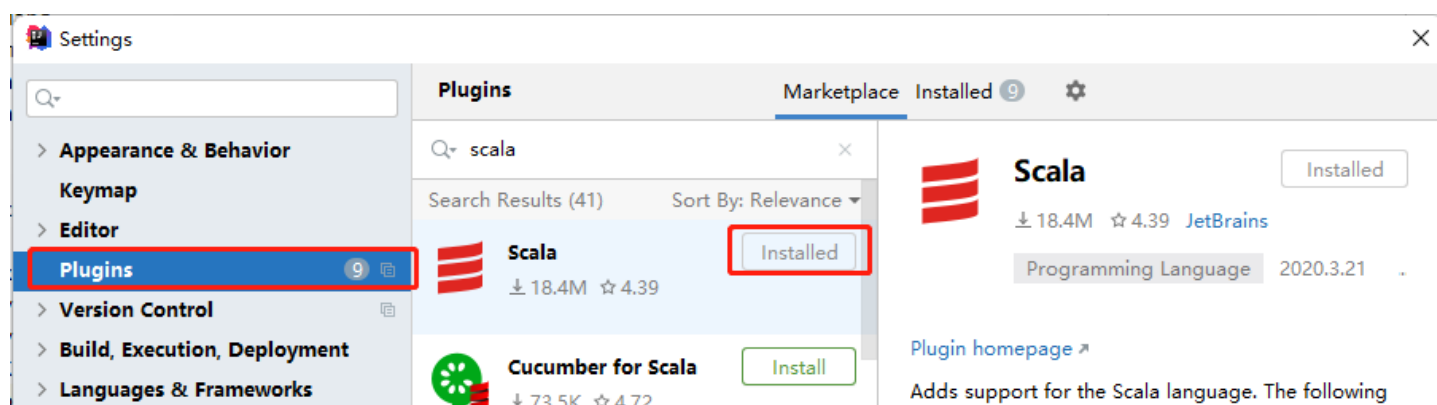
八、实现

kyuubi开发环境准备

idea配置

https://kyuubi.apache.org/docs/r1.6.0-incubating/develop_tools/idea_setup.html

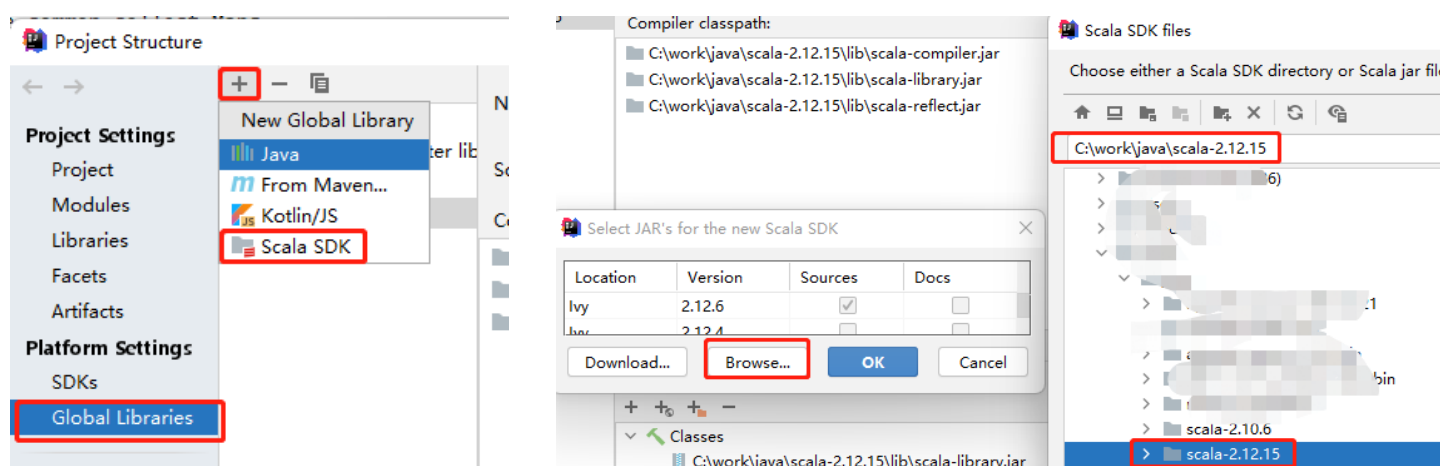
1. 每个文件头都有Apache的License
2. 安装Scala插件



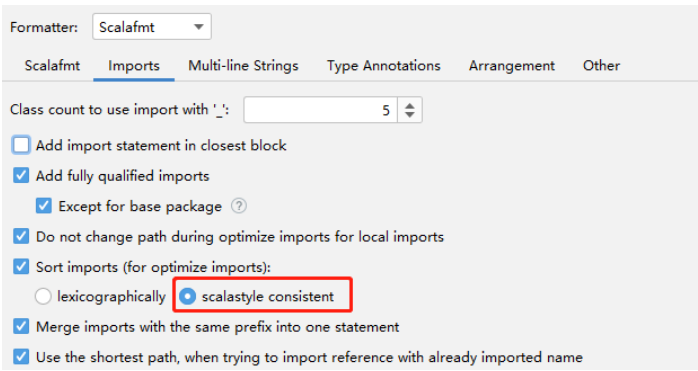
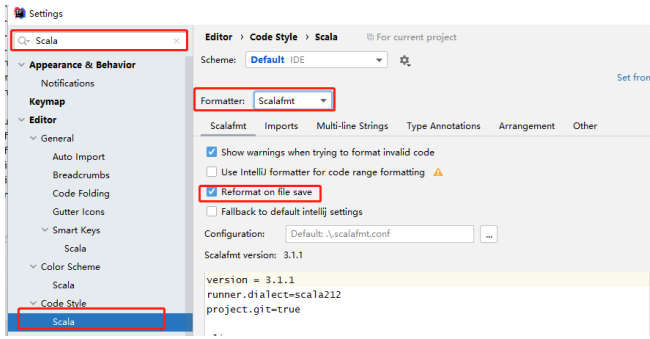
3. 配置Scala环境



下载，解压scala包。配置Scala SDK



1. 配置Scala format



shell任务类型

 [kyuubi-batch-podtask.postman_collection.json](#)

配置参数映射

参数描述	kyuubi参数	对应的linkis参数
镜像地址	kyuubi.batchConf.pod.image	
pod运行的namespace，需 要与kyuubi配置的kubefconfi 地址一致	kyuubi.batchConf.pod.namespace	
cpu 下限	kyuubi.batchConf.pod.resource.request.cpu	shell.client.request.core
memory下限	kyuubi.batchConf.pod.resource.request.me mory	shell.client.request.memor y
cpu上限	kyuubi.batchConf.pod.resource.limit.cpu	shell.client.core
内存上限	kyuubi.batchConf.pod.resource.limit.memo ry	shell.client.memory
额外存储容量（默认单位为 G）	kyuubi.batchConf.pod.extra.disk.volume	shell.extra.disk.volume

任务参数

参数名	含义
--executionCode	shell的代码块
--resources	资源文件
--glodon-user-token-id	

	linkis的token，用于下载任务的资源文件
--dolphin_process_id	海豚任务流id，用于日志记录
--task_center_instance_id	任务中心实例id，用于日志记录
--task_id	任务id，用于日志记录
--dolphin_process_instance_id	海豚任务流实例id，用与日志记录

运行任务

```

1 请求
2 POST /api/v1/batches HTTP/1.1
3 Host: 192.168.127.164:10099
4 Content-Type: application/json
5 Content-Length: 1061
6 {
7   "batchType": "shell",
8   "resource": "none",
9   "className": "none",
10  "name": "shell",
11  "conf": {
12    "kyuubi.batchConf.pod.image": "registry.cn-beijing.aliyuncs.com/gldsg-
    pdc/shell-engine:tmp-dev-1a6e8944e-v4",
13    "kyuubi.batchConf.pod.namespace": "linkis-dev",
14    "kyuubi.batchConf.pod.resource.request.cpu": "10m",
15    "kyuubi.batchConf.pod.resource.request.memory": "10M",
16    "kyuubi.batchConf.pod.resource.limit.cpu": "1",
17    "kyuubi.batchConf.pod.resource.limit.memory": "30M",
18    "kyuubi.batchConf.pod.extra.disk.volume": "20"
19  },
20  "args": [
21    "--executionCode","set -e \n pwd\n echo \"aaa\"\n for i in
    {1..100000};do\n head -n 10  /dev/urandom | od -x | sed 's/\\s*//g' \n sleep
    0.1\n done;",
22    "--resources", "[{\"resourceId\":\"1293e12f-8452-4389-8b07-55d6ce7634d5\",
    \"fileName\": \"kettle/http_test.ktr\"}]",
23    "--glodon-user-token-id", "token_liurx_debug",
24    "--dolphin_process_id", "1",
25    "--task_center_instance_id", "2",
26    "--task_id", "3",
27    "--dolphin_process_instance_id", "4"
28  ]
29 }
```



```
30
31 响应结果
32 {
33     "id": "0d23f3a6-b10b-4c29-be74-cafb55160769",
34     "user": "anonymous",
35     "batchType": "SHELL",
36     "name": "shell",
37     "appId": null,
38     "appUrl": null,
39     "appState": "NOT_FOUND",
40     "appDiagnostic": null,
41     "kyuubiInstance": "192.168.127.164:10099",
42     "state": "PENDING",
43     "createTime": 1666927651740,
44     "endTime": 0
45 }
```

停止任务

```
1 请求
2 DELETE /api/v1/batches/2d1d0131-1d9c-4ae2-a107-aa70f0cd6883 HTTP/1.1
3 Host: 192.168.127.164:10099
4
5 结果
6 {
7     "success": true,
8     "msg": "Operation of deleted appId: pod-2d1d0-202210281126 is completed"
9 }
```

获取任务状态

```
1 请求
2 GET /api/v1/batches/0d23f3a6-b10b-4c29-be74-cafb55160769 HTTP/1.1
3 Host: 192.168.127.164:10099
4
5 结果
6 {
7     "id": "0d23f3a6-b10b-4c29-be74-cafb55160769",
8     "user": "anonymous",
```

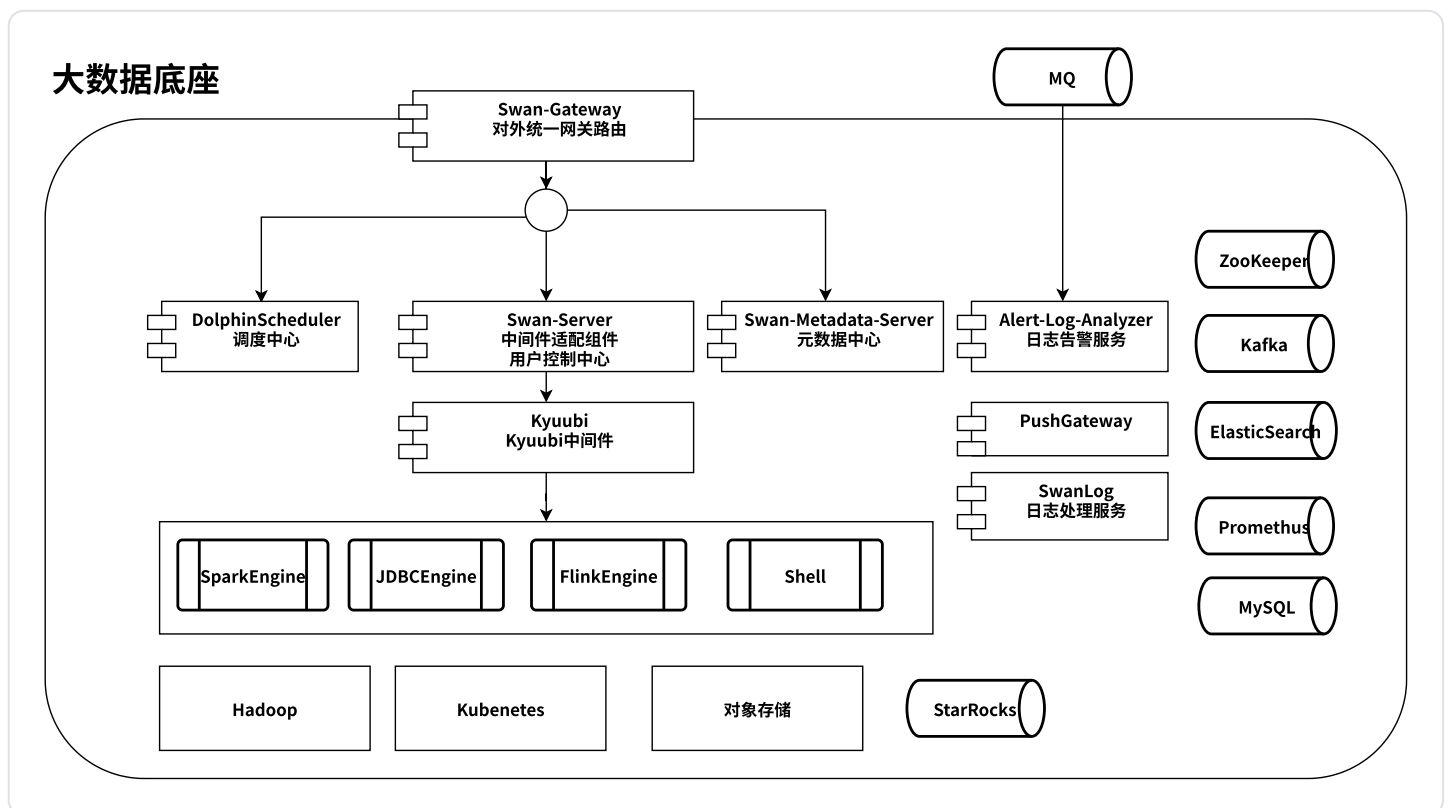
```

9     "batchType": "SHELL",
10    "name": "shell",
11    "appId": "3627d7ef-3373-4521-a230-4d2fafa95139",
12    "appUrl": null,
13    "appState": "FINISHED",
14    "appDiagnostic": null,
15    "kyuubiInstance": "192.168.127.164:10099",
16    "state": "FINISHED",
17    "createTime": 1666927651740,
18    "endTime": 1666939926434
19 }

```

九、部署

物理架构



kyuubi构建

官方的代码中提供两种构建镜像的方式

1、在docker镜像中进行编译，用dist脚本构建，把构建结果放到最终的镜像中【官方的镜像，应该是使用该方式】

2、在本机进行编译，用bin/docker-image-tool.sh构建最终镜像【需要修改build/Dockerfile脚本，增加flink等其余的引擎】 【适合本地调试】

方式1: build/Dockerfile

下载spark、flink等安装包较慢，即使用了mirror-cdn

```
1 docker build \  
2   --build-arg MVN_ARG="-DskipTests -Drat.skip=true -Pmirror-cdn" \  
3   --file build/Dockerfile \  
4   --tag registry.cn-beijing.aliyuncs.com/gldsg-pdc/kyuubi-dist:debug .
```

方式2: docker/Dockefile

1. 编译

```
1 ./build/mvn clean package -DskipTests
```

2. 构建docker镜像

```
1 # 对docker/Dockerfile的修改  
2 # 1. 去掉拷贝beeline-jar的步骤  
3 # 2. 增加阿里云的apt源  
4 sudo KYUUBI_VERSION=1.6.0-incubating KYUUBI_HOME=/home/liurx/code/gdmp-middle-  
kyuubi /bin/bash -x /home/liurx/code/gdmp-middle-kyuubi/bin/docker-image-  
tool.sh -r registry.cn-beijing.aliyuncs.com/gldsg-pdc -t 1.6.0-incubating -f  
/home/liurx/code/gdmp-middle-kyuubi/docker/Dockerfile build  
5  
6 # 可以指定spark从另外一个base镜像中获取  
7 sudo KYUUBI_VERSION=1.6.0-incubating KYUUBI_HOME=/home/liurx/code/gdmp-middle-  
kyuubi /bin/bash -x /home/liurx/code/gdmp-middle-kyuubi/bin/docker-image-  
tool.sh -r registry.cn-beijing.aliyuncs.com/gldsg-pdc -t 1.6.0-incubating -f  
/home/liurx/code/gdmp-middle-kyuubi/docker/Dockerfile -b  
BASE_IMAGE=registry.cn-beijing.aliyuncs.com/gldsg-pdc/spark:v3.3.0-seaweedfs -  
S /opt/spark build
```

统一存储

[illegible]

[illegible]

[illegible]

[illegible]